QWEN3.8-27B | ENGINE RESEARCH SERIES

vLLM 0.32.0 Research

বাংলা গবেষণা ডসিয়ার

Kaggle 4× NVIDIA L4 · Ada SM89

Source-backed experiments, correctness gates এবং measurable priorities

1500+ tok/s

Sustained aggregate accepted-output decode · research target

vLLM 0.32.0 | MODEL: Qwen3.8-27B | P0 / P1 / P2 / P3 | OFFLINE KAGGLE

VERSION 5.0 · 29 OCT 2026

Evidence-graded · URLs clickable

সূচিপত্র / Research sections

- Executive decision: এক নজরে
- A. Locked environment, weight formats, model architecture
- B. Actual engine configurations (do not mix FP8 and INT4 gates)
- C. Measurement ledger — exact comparability
- D. P0 — accuracy, validity and instrumentation gates (mandatory)
- E. P1 priority experiments — run in this order
- F. P2 medium-upside experiments
- G. P3 research-only, higher-risk architecture changes
- H. Experimental design — cheapest-to-expensive
- I. Tested-negative branches — avoid repeated GPU waste
- J. Related Kaggle 4×L4 references (deployment, not model decode evidence)
- K. Source register — raw checkable URLs for Claude
- L. Update protocol — planned integration of external AI research
- L. V2.0 সংযোজন: AI Research 01, দুটি পূর্ণ গবেষণার source-preserving integration
- M. V-P1: FP16 → BF16 GDN state: one-flag, kernel-selection experiment
- N. V-P1: FULL speculative verification graphs and capture-grid research
- O. Hybrid prefix-cache: partial match, shared junction, finalized continuation
- P. Merlin Engine: 4×TP1 research, memory layout, and five source-level patches
- Q. FP8 SM89 native CUTLASS, KV and memory pool
- R. Adaptive speculative decoding and EAGLE-4
- S. Hardware feasibility, agent scheduling, and Kaggle adjacent results
- T. Revision audit 01 — source coverage, conflicts, and evidence gaps
- W. দ্বিতীয় AI-এর গবেষণা — নতুন source-backed experiments ও 1500+ অভিযানের অগ্রাধিকার
- X. Version 3 — দ্বিতীয় AI গবেষণার পূর্ণ coverage audit
- Y. Version 4 — তৃতীয় AI-এর সম্পূর্ণ গবেষণা সমন্বয়: vLLM 0.32.0
- Z. v4 audit ledger — তৃতীয় AI গবেষণার পূর্ণ coverage এবং source status
- AA. Final integration v5: Claude E22 ও পাঁচটি গবেষণার সমন্বিত সিদ্ধান্ত

বাংলা Research Dossier — Version 5.0 | 29 October 2026 | FINAL FIVE-SOURCE INTEGRATION
Mission: Sustained aggregate accepted-output decode **1500+ tok/s**; ultimately more correctly submitted ARC answers per 12-hour Kaggle run.

> **Scope:** Qwen3.8-27B + Kaggle 4×L4; vLLM 0.32.0 is the reference codebase, **not a frozen configuration**. TP1/TP2/TP4/PP4, quantization, dtype, speculation, CUDA graph, fork/backport and model-local memory layout are all legitimate experiments when independently tested.

SGLang findings are in the separate PDF. vLLM 0.19 and Qwen3.8-Flash-Next are **out of scope**. NVIDIA Team-এর 1500+ performance-কে গবেষণার user-specified target হিসেবে গ্রহণ করা হয়েছে; এ নথি সেটির independent proof দাবি করছে না।

Reading key. [LOCAL] = আপনার সংরক্ষিত EMS/VMS/Kaggle GPU measurement; **[UPSTREAM]** = linked issue/PR/docs-এ অন্য environment-এর evidence; **[HYPOTHESIS]** = আমাদের hardware-এ এখনও পরীক্ষা হয়নি; **[GATE]** = production-এর আগে বাধ্যতামূলক test। কোনো public uplift-কে 4×L4-এ predicted gain হিসেবে ব্যবহার করা হয়নি।

V4 CURRENT RESEARCH POLICY. 1500+ aggregate tok/s আমাদের target। অন্য GPU/engine fork-এর implementation হলো mining source: profiler ও patch-এর mechanism আলাদা করে 4×L4-এ port/test করা হবে। Current baseline stable reference, বাধ্যতামূলক স্থায়ী configuration নয়। Original V1+V2 sections preserved; new research and full tests added in W/X; follow **AA1 Version 5 final priority queue** first; Y1 remains the historical v4 campaign.

V5 FINAL RESEARCH NAVIGATOR. নতুন AI-এর দুইটি research segment বিশ্লেষণ করে **Y1**-এ নতুন highest-priority experiment matrix, **Y2–Y11**-এ implementation/negative tests এবং **Z**-এ complete insight-coverage audit যোগ হয়েছে। **Y1** আগে পড়ুন। V1/V2/V3-এর সব substantive sections ও primary URLs সংরক্ষিত; existing configs চলমান reference, freeze নয়।

VERSION 5 FINAL EXPERIMENT INDEX: প্রথমে **AA1** (new final priority order), তারপর **AA2–AA9/AA10** (Claude E22 raw NCCL evidence, source-backed code candidates, unresolved risks, five-study coverage ledger) পড়ুন। পুরোনো v1–v4-এর সব বিস্তারিত finding, raw URL এবং experiment priority history রাখা হয়েছে। Engine configs reference control, অপরিবর্তনীয় সিদ্ধান্ত নয়। 1500+ aggregate tok/s আমাদের user-specified research target।

Executive decision: এক নজরে

1. **P0: reliability-first:** INT4-এর নিজস্ব target-only oracle এবং FP8 baseline ধরে cold/warm/MTP/concurrency/state-continuity gate; নইলে দ্রুত কিন্তু silent-wrong engine গ্রহণ করা যাবে না।
2. **P1: high-confidence workload value:** `--prefix-match-unit 16/32` + `--enable-mamba-shared-prefix-checkpoint` দিয়ে actual multi-turn prefix reuse উন্নত হয় কি না পরীক্ষা। এটি decode kernel speed নয়; real agent throughput বাড়ানোর প্রার্থী।
3. **P1: critical-path compute:** PCIe/SHM NCCL, FlashInfer-vs-Triton attention + effective CUDA graph, TP-wide sampling/all-gather, GDN verify path; measured bottleneck।
4. **P1 gated:** speculative last-block preservation (`disable_eagle_block_drop=true`) — public benchmark বড় লাভ দেখালেও wrong-state risk আছে।
5. **P2/P3:** prefill/decode starvation, DBO, active-seq cap, Ada native FP8 kernel, Mamba-state dtype, PP4; আগে bottleneck profile, পরে backport।

A. Locked environment, weight formats, model architecture

উপাদান	স্থির তথ্য / implications
GPU/topology	4× L4, 24 GB each, Ada SM89, PCIe only; P2P/NVLink guarantee নেই; default collective path PYNCCL SHM।
Notebook	Offline Kaggle; Python 3.12 image; torch 2.10 cu128 base; vLLM 0.32.0 cu13 isolated venv; 12-hour hard limit; startup <~10 min।
Model	Qwen3.8-27B; `Qwen3_5ForConditionalGeneration`; 48 GDN + 16 full attention layers, one MTP head; large vocabulary.
FP8	Block-FP8 repack; block 128; actual SM89 linear kernel `MarlinFP8ScaledMMLinearKernel`; MTP3।
INT4	RedHat compressed-tensors W4A16 group128 / `pack-quantized` / static actorder; GDN selected projections quantized, MTP head and lm_head BF16; `MarlinLinearKernel`।
Cache	FP8 e4m3 KV; hybrid Mamba align prefix checkpoint; under long context actual running streams < requested。
Offline	No runtime pip install from Internet: wheelhouse + any patch/JIT cache/kernels must be staged into Kaggle Dataset, SHA verified।

Artifact ground truth: `Q27_optimize_vLLM032_SGLang_2026-10-29.zip` → `vllm_032/README_vllm032.md`, `vllm_032/results/vms_report.md`, server logs, E21 smoke reports; `Q27_best_vLLM_SGLang_notebooks_2026-10-29.zip` actual NB1/NB2 notebooks। These are LOCAL reports, not independently rerun GPU results in this document.

B. Actual engine configurations (do not mix FP8 and INT4 gates)

B1. FP8 correctness baseline — VMS `fp8_mtp3_ssm_fp16`, 16 streams

```
vllm serve "$FP8_MODEL" \
  --tensor-parallel-size 4 \
  --kv-cache-dtype fp8_e4m3 \
  --max-model-len 106496 \
  --max-num-seqs 16 \
  --max-num-batched-tokens 4096 \
  --gpu-memory-utilization 0.90 \
  --disable-custom-all-reduce \
  --attention-backend FLASHINFER \
  --enable-prefix-caching \
  --mamba-ssm-cache-dtype float16 \
  --speculative-config '{"method":"mtp","num_speculative_tokens":3}'
```

This is a **reconstructed core command** from the VMS flag builder, not a replacement for the complete notebook-generated argv. Exact staged model path, served name, environment, seed, `--language-model-only`, and compatibility options must come from the generated launch manifest.

Runtime note: fp16 recurrent state on vLLM 0.32 selects **Triton GDN spec-decode fallback**, whereas default BF16/FP32 selects fused CUDA GDN decode. Despite kernel fallback, fp16 reached more XL streams through lower memory footprint. Do not infer fastest kernel from dtype name alone.

B2. INT4 throughput challenger — VMS `int4_mtp3_s20`

Same TP4/FlashInfer/FP8-KV/MTP3 architecture; switch weights to INT4, request 20 streams, use actual SSM default from arm. **Coherent and fast, but NOT INT4-oracle-certified.** Its 0.19 failure does not transfer to 0.32; conversely 0.32 coherence alone does not prove ARC accuracy.

B3. Full 135k Duck half-B smoke

E21 vLLM 0.32 experimental half-B: `--max-model-len 135168 --language-model-only`, INT4 S20 in NB1 commit; FP8 SSM fp16 S16 in NB2. A single E21 half-B smoke p50 ≈322 (INT4) / 329 (FP8); prefix hits ≈56% / 55%. These are **short diagnostic smoke metrics**, not VMS decode-only or LB measurements. E21 NB1 hidden rerun uses its **half A**, not this 0.32 half B; do not present half-B as an achieved hidden-LB score.

C. Measurement ledger — exact comparability

VMS arm	8k output tok/s	40k	~90k XL	XL actual running	Quality evidence
FP8 MTP3 S16	434	399	281	10	FP8 target-only boundary gate PASS
FP8 MTP3 S20	452	423	281	10	boundary PASS
FP8 MTP3 S24	472	424	256	9	boundary PASS; **worse XL**
FP8 MTP3, SSM fp16 S16	**450**	**414**	**312**	11	FP8 boundary PASS
INT4 MTP3 S16	495	434	321	12	diagnostic; oracle absent
INT4 MTP3 S20	**518**	**439**	**332**	12	diagnostic; oracle absent
FP8 MTP2	448	397	260	10	no general win
FP8 MTP4	399	381	267	9	no general win
FP8 + PCIe-IPC	297	278	206	10	only ~0.68–0.73×

Critical measurement nuance: XL test requested streams ≠ active running streams. The 90k entries correspond to an ~90.6k prompt band whose XL occupancy is partially shaped by KV capacity and shared-prefix benchmarking. They do **not** demonstrate 16–24 independent resident 90k requests. 40k contains shared ~36k system prefix, while prefix-off arms re-prefill; `tok/s` is a **steady decode-line** metric after prefill settles, not real agent solve/hour.

Throughput gap to target

- vLLM INT4 XL **332 → 1500** = **4.52×** required.
- vLLM FP8 SSM fp16 XL **312 → 1500** = **4.81×** required.
- For approximate aggregate accepted output throughput: `TPS ≈ actual_running × accepted_tokens_per_step × 1000 / critical_path_step_ms`. At **12 running × 2.92 accepted tokens**, 1500 requires ~**23.4 ms/step**. This arithmetic is **not a predicted engine improvement**; it exposes why modest kernel tuning cannot alone meet target.
- Track `accepted_tokens` not speculative draft tokens; warm vs cold, complete vs incomplete, distinct vs shared prefixes separately.

D. P0 — accuracy, validity and instrumentation gates (mandatory)

V-P0-01: FP8 baseline no-regression

Purpose: preserve passing FP8 MTP3 reference under every performance change.

Protocol: 16–32 frozen ARC prompts, 1/8/16/20 concurrent, greedy serial/cold/warm, 512–2048+ generated tokens, plus 8k/40k/90k/130k long states. Compare target-only no-prefix to each MTP arm, same checkpoint/tokenizer/input IDs. Record first divergent token, top-two logprob margin, repeating punctuation, early EOS, acceptance and finish reasons. Near-tie drift alone ≠ corruption. No `PASS` when logprobs, expected pairs or clean cold reference missing.

Fail: high-margin output flip; empty/garbage tool call; wrong-state cache restore; incomplete evidence mislabeled PASS; stream error, OOM or fallback to different model. **Only after this gate** run paired ARC solve tasks with equal budgets.

V-P0-02: INT4 must have INT4 target-only oracle

Why: VMS FP8 oracle does not certify INT4. Create INT4 target-only, prefix OFF, deterministic eager/reference mode on 0.32 and compare INT4 MTP3+caching to it; additionally compare INT4 vs FP8 task-level score (not token exact). Store exact weight hashes and quantized FP8 KV scales. Candidate W4A16 fast arm remains diagnostic until its own oracle PASS and ARC non-regression.

V-P0-03: Forced rejection and recurrent-state safety

Upstream vLLM **#63593** demonstrated Qwen3.8 MTP **K≥5**, concurrency≥8, L≥192 confident target-only divergence. Their K3 control did **not** reproduce fail-class behavior; **do not claim K3 is known-broken**. Nevertheless, force rejection at K3, capture accepted/committed length, compare target-only logits/states at boundaries and after long rejected runs. Re-run after any cache/block-drop, CUDA-graph or dtype change.

Source: https://github.com/vllm-project/vllm/issues/63593

V-P0-04: Prefix match must exercise generated continuation, not just prompt

Report `prompt_tokens`, `cached_tokens`, `cached_generated_tokens` and match boundary. If a suspected MTP trailing-block reuse path is never exercised, verdict for that path is `NOT_EXERCISED`, **not PASS**. Cold reference must verify flush/idle and `cached_tokens=0`; warm reference must verify expected hit. Run 5+ repeated multi-turn cycles with tool-call additions.

V-P0-05: Valid benchmark metrics and engine readiness

Record `effective_graph_mode`, resolved max-running, number of actual live streams, queue/prefill interference, preemptions, GDN decode kernel selected, warmed-up JIT, model/mamba weights integrity, no unexpected backend fallback, startup <600s, no poisoned driver cache. A healthy `/health` alone is not a generation progress test. Preserve partial data on a crash.

E. P1 priority experiments — run in this order

V-P1-01 — Fine-grained hybrid prefix keys (first lightweight arm)

Rationale [UPSTREAM]: Hybrid GDN aligned block can be ~1.6k tokens; one dropped block per multi-turn prefix hit becomes repeated expensive prefill. vLLM 0.32 supports a finer matching key interval, but keys alone are **not sufficient to guarantee saved Mamba state**. The model's resolved block sizes must be divisible by the chosen unit.

A/B ladder:

```
# A0 current 0.32 FP8 MTP3
# A1 ONLY:
--prefix-match-unit 16
# A2 on top of A1:
--enable-mamba-shared-prefix-checkpoint
# A3 if all KV groups permit it: A2 with --prefix-match-unit 32
```

Measure: matched agent-turn cached/new tokens, TTFT, 20-minute mixed-loop accepted output TPS, actual running streams, mamba checkpoint memory and preemptions. Compare 16/32/default; validate startup divisibility and flag activation. No user-message/task changes.

Promote: >10% matched real loop improvement with no correctness regression; lower than this may still be useful but investigate launch/overhead first. **Evidence:** version-pinned docs https://docs.vllm.ai/en/v0.32.0/features/automatic_prefix_caching/ and #53670 https://github.com/vllm-project/vllm/issues/53670

V-P1-02 — Trailing speculative block retention (highest-risk P1)

```
--speculative-config '{"method":"mtp","num_speculative_tokens":3,"disable_eagle_block_drop":true}'
```

Upstream reported, NOT local: #53670 no-spec 327; DFlash K7 default block-drop 206; drop disabled 322 tok/s on TP2×5090 repeated-prefix workload. Another K2 repeated-turn report 149.6→241.9 (+62%). **These are warm-turn throughput effects, not pure 90k decode boost.**

Risk: Last block is intentionally discarded to avoid stale/speculative KV/Mamba state; blanket disabling can create silent corruption. Test successor token equality, committed length, forced rejection, cold/warm/serial/16–20 concurrent, multi-turn, tool-call validity and ARC tasks. A2 default-block-drop is the matched control. Never adopt on throughput result alone.

Sources: https://github.com/vllm-project/vllm/issues/53670 ; https://github.com/vllm-project/vllm/pull/53388 ; https://github.com/vllm-project/vllm/issues/53912

V-P1-03 — NCCL SHM collective critical-path tuning

LOCAL: a TP4 step profile in the shared pack had 50.9 ms all-reduce plus 8.1 ms all-gather within a 116 ms step; confirm **critical-path** rather than sum of overlapping GPU events. Default SHM/PYNCCL ~2 channels; PCIe-IPC failed with ~0.68×. Forcing P2P on this topology was much slower in existing microbenchmark. Do not retry unsupported all-reduce blindly.

Low-risk isolated matrix:

```
# per process launch, not edited inside a running server
NCCL_MIN_CTAS=1 NCCL_MAX_CTAS=1
NCCL_MIN_CTAS=2 NCCL_MAX_CTAS=2
NCCL_MIN_CTAS=4 NCCL_MAX_CTAS=4
NCCL_MIN_CTAS=8 NCCL_MAX_CTAS=8
# optional controlled settings only if supported by linked NCCL version
NCCL_ALGO=Ring   # vs NCCL_ALGO=Tree
NCCL_NTHREADS=512
```

Benchmark same 1MB-class all-reduce under graph/capture and **full model 40k/90k**. Record NCCL version, transport, channel count, critical-path latency, GPU occupancy and TPS. Some CTA counts may harm GEMM occupancy; reject negative full-model outcomes despite faster microbench. Source: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

V-P1-04 — Effective CUDA graphs / attention backend in MTP verification

LOCAL: FP8 engine's requested `FULL_AND_PIECEWISE` falls to `PIECEWISE` with FlashInfer speculative verify. Test `FLASHINFER` vs `TRITON_ATTN` plus supported compilation-config modes; read actual selected mode from logs, not requested flags. Track graph sizes for active B×(K+1), graph memory, cold-start compilation, verify-step wall time.

Source candidate [UPSTREAM, not installed in 0.32]: https://github.com/vllm-project/vllm/pull/60532 reported alternative FlashInfer multi-token verify route. Check exact source diff, wheel ABI and compatibility before backport. New code is **not automatically present** in pinned 0.32. Installation+startup <10min must hold. Design: https://github.com/vllm-project/vllm/blob/main/docs/design/cuda_graphs.md

V-P1-05 — SSM dtype, fused GDN kernel versus active-sequence capacity

Existing result: fp16 SSM **450/414/312** vs default FP8 **434/399/281** at S16; fp16 GDN speculative path falls back to Triton, but +1 XL active stream improves aggregate throughput. Test default BF16, explicit FP16, and FP32 only if VRAM allows. Capture GPU kernel routing, Mamba cache slots, active streams, GDN state parity and long-run precision. A dtype that speeds kernels but drops concurrency can lose overall TPS.

V-P1-06 — Proper active-stream and admission-cap sweep

At 90k requested S24 delivered 9 active, worse than S16 with 10. Test S12/S16/S20/S24 with actual `running_p50`, `running_p90`, preemption and prefix ownership. If supported by the installed v0.32 flags, separate maximum graph/sequences allocation from actual admitted active cap (`--max-num-active-seqs`) and compare. Prefer dynamic agent-side scheduling that keeps useful short/medium work resident while avoiding 90k KV oversubscription. **No assumption that higher max_num_seqs → higher TPS.**

F. P2 medium-upside experiments

V-P2-01 — Prefill/decode interference and cache-preserving multi-turn

Issue https://github.com/vllm-project/vllm/issues/54919 includes **other GPUs/versions**, even an Ada SM89 Qwen3.8-27B report of extreme decode starvation during two cold long prefills; not proven on this TP4 setup. Run D1 resident decode; D2 16 resident + one cold 90k prefill; D3 real tool-turn trace. Sweep `--max-num-batched-tokens 2048/4096/8192` with controlled active seq count; observe per-step histogram, no OOM/restart. Preserve deterministic prefix tokens and reuse between turns. Candidate `--long-prefill-token-threshold 8192` requires installed help check: it is not a magic decoder-priority guarantee.

V-P2-02 — Dual batch overlap (DBO)

`--enable-dbo` is an upstream-documented experimental vLLM option. Test target-only clean short first, then MTP3, then 40k/90k mixed; verify all request outputs and tool calls. If launch doesn't support Qwen hybrid path or graph overlap alters state, reject. Expect unproven gain on 4×L4; use source/CLI preflight.

V-P2-03 — Batch-sharded sampling / avoid vocab all-gather

The 248k-vocab LM-head/logit processing and TP4 all-gather can consume several ms/step. `--enable-batch-sharded-sampling` exists in upstream docs, but only works if this Qwen model implements `compute_logits_local`. **Before GPU:** inspect installed `Qwen3_5ForConditionalGeneration` for method and flag. If unsupported, do not launch a no-op. A code patch needs explicit token-ID/logprob equivalence tests and proper TP-group synchronization.

V-P2-04 — Native Ada FP8 W8A8 and quantized matrix path

SM89 block-FP8 currently selects **MarlinFP8ScaledMM**; existing `--linear-backend triton` measured **438/398** vs **434/399** baseline, not a big win. Source issue https://github.com/vllm-project/vllm/issues/58241; experimental CUTLASS SM89 branch https://github.com/vllm-project/vllm/pull/61224 (pack note: single-L4 Qwen3-8B measured uplift, mostly prefill; not your kernel). A source build must preserve FP8 block scales, MTP projection, sm89 and offline wheels. Reject if launch >10min or target-only parity fails.

V-P2-05 — CPU affinity, request orchestration, tool-call overlap

With 48 vCPU, agent tool calls can serialize or starve API producers while GPU sits idle. Measure utilization/time decomposition `prefill + verify + tools + queue + final-drain`. Avoid releasing large cache residency after every short tool call; for stable task prefix use same engine session and deterministic bytes. Tune CPU workers and admission with GPU trace; don't mistake benchmark decode-only TPS for 12-hour accepted solve throughput.

G. P3 research-only, higher-risk architecture changes

V-P3-01 — Pipeline parallel PP4 instead of TP4

Candidate `--tensor-parallel-size 1 --pipeline-parallel-size 4` may reduce per-layer PCIe collectives but introduces pipeline bubbles, imbalance and MTP compatibility/workspace constraints. Begin target-only INT4/FP8 8k smoke; then 40k/90k; then MTP3. Verify stage-wise GDN cache allocation, throughput & startup. Not a proven drop-in; if unsupported, stop early. Community discussion: https://github.com/vllm-project/vllm/issues/60765

V-P3-02 — NVFP4 KV / more resident XL concurrency

Candidate PR https://github.com/vllm-project/vllm/pull/46963 may improve KV capacity in other hardware. SM89 unverified in source pack; possible numerical loss and decode speed regression. Do **not** assume `--kv-cache-dtype nvfp4` accepted or fast on L4; validate exact implementation, calibration, attention kernel, and full-target parity before benchmarking.

V-P3-03 — Mamba decode checkpoints and ReplaySSM

PR https://github.com/vllm-project/vllm/pull/50551 proposes better cached continuation; source pack reports 63.6→94.7% prefix hit under a different stack. Do not conflate with existing 0.32 fine-grained flags. Version-pinned docs also describe `--use-replayssm`; Mamba2/FlashInfer backend and

prefix mode conditions are restrictive. A Qwen3.8 GDN/TP4/MTP3 compatibility check is required; don't steal a SGLang replay flag and assume it maps to vLLM. Measure real prefix reuse, state correctness and memory, not only single-layer microbench.

H. Experimental design — cheapest-to-expensive

Stage	Included arms	Data / runtime criterion	Stop or escalate
0: offline CPU preflight	installed `--help`, class methods, PR diff hash, checkpoint/config hashes, exact argv	no GPU	unsupported flag → reject arm
1: 8k smoke	A0 + prefix16 + shared checkpoint; three numeric+JSON/tool-call probes	warm ready <600s	wrong first token / graph missing → stop
2: 40k shared-loop	A0/A1/A2/A3; same 4 turns, 8–20 streams	cached-tokens + wall accepted TPS	>10% matched gain → keep
3: decode critical path	NCCL & graph pair; compare 20min windows	P50/P90 step/collective, accepted/verify	faster microbench but slower engine → reject
4: 90k/130k stress	best 2–3 arms, shared and distinct	actual streams, preemption, throughput	OOM, wrong-state, long stall → reject
5: task outcome	paired 24–64 ARC tasks, fixed token/wall budgets	solve-rate with uncertainty	meaningful accuracy regress → NO-GO
6: 12h equivalent	same production solver/ harness, cutoff/drain, JSON serialization	correct **submitted** answers / hour	only end-to-end winners promoted

Standard benchmark dimensions

- **D1:** resident decode-only; all prompts prefetched, no new prefill; first measure engine kernel ceiling.
- **D2:** shared-prefix 4–8 agents/task, append tool-feedback turns; captures cache opportunity.
- **D3:** distinct 50k/90k/130k prompts, true capacity, mixed prefill and decode starvation.
- Each arm: at least 3 A/A repetitions where feasible, same frozen inputs/seed, CPU timer around complete run, same active concurrency **reported** not only requested. Include no-progress watchdog, GPU peak memory, error counts, completed-task coverage, exact accepted-output tokens.

```
# schema: placeholders are NOT observations
record = {
 'engine':'vllm-0.32.0','arm':'prefix16_shared',
 'model_sha':'...', 'wheel_sha':'...', 'args_sha':'...',
 'requested_streams':20, 'running_p50':None,
 'prompt_p50':None, 'cached_tokens':None,
 'accepted_output_tokens':None, 'wall_seconds':None,
 'aggregate_tps':None, 'accept_per_verify':None,
 'collective_critical_path_ms':None,'verify_step_ms':None,
```

```
  'preemptions':None, 'wrong_outputs':None,
  'startup_seconds':None, 'target_oracle_pass':False,
  'arc_nonregression_pass':False,
}
```

Fail-closed promotion rule

A new engine is **PRODUCTION GO** only when: exact source and weights match report; clean target-only reference exists; expected comparison pairs/logprob evidence complete; cold/warm/cache-specific gates pass; no cross-request contamination, repeating punctuation, premature EOS, systemic stall, OOM or new numerical high-margin flips; actual 90k/130k decode and mixed agent throughput improve; paired ARC accuracy does not materially regress; startup and final-drain budgets pass. Otherwise **DIAGNOSTIC ONLY** or **NO-GO**.

I. Tested-negative branches — avoid repeated GPU waste

Rejected / de-prioritized	Local evidence
PCIe-IPC all-reduce	FP8 8k 434→297, 40k 399→278; ~0.68–0.70×.
TP2×2 data parallel	At 40k, actual streams collapsed to 2; KV-limited.
MTP2/MTP4 vs MTP3	40k performance no better, XL worse.
Prefix caching OFF as optimization	40k arm collapsed to ~3 tok/s / only 2 active, due full reprefill; output drift risk separate.
`--linear-backend triton` as FP8 miracle	438/398 vs reference 434/399; essentially parity.
Nonexistent old `VLLM_TEST_FORCE_FP8_MARLIN` flag	Removed in v0.32; silently invalid experimental arm.
vLLM 0.19 INT4 loader rescue	Out of scope; unrelated to 0.32 winner.
Huge MTP K / unguarded trailing block	Upstream wrong-state reports; cannot maximize speed by skipping quality controls.

J. Related Kaggle 4×L4 references (deployment, not model decode evidence)

Other Kaggle competitions offer staging, quantized model load and offline wheel ideas, not measured Qwen3.8-27B TP4 90k speed:

- ARC Prize 2026 ARC-AGI-2: https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2
- ARC Prize 2025: https://www.kaggle.com/competitions/arc-prize-2025
- Gemma 4 Developer Agent: https://www.kaggle.com/competitions/gemma-4-developer-agent
- Konwinski Prize: https://www.kaggle.com/competitions/konwinski-prize
- 72B AWQ on 4×L4 public notebook: https://www.kaggle.com/code/abdullahmeda/load-72b-awq-model-using-vllm-on-l4-x4

K. Source register — raw checkable URLs for Claude

All links below are explicit full URLs and clickable PDF hyperlinks. Current-upstream source may have changed after pinned 0.32.0; compare **tagged source** and installed wheel before assuming a fix is available.

[V01] vLLM v0.32 exact CLI: https://docs.vllm.ai/en/v0.32.0/cli/serve/

[V02] v0.32 automatic prefix caching / shared checkpoint: https://docs.vllm.ai/en/v0.32.0/features/automatic_prefix_caching/

[V03] One hybrid block dropped on each speculative cache hit: https://github.com/vllm-project/vllm/issues/53670

[V04] Opt-out block-drop implementation: https://github.com/vllm-project/vllm/pull/53388

[V05] Hybrid MTP + prefix-cache corruption reports: https://github.com/vllm-project/vllm/issues/53912

[V06] K≥5 forced reject wrong state in v0.32: https://github.com/vllm-project/vllm/issues/63593

[V07] Qwen hybrid cold-prefill/decode starvation: https://github.com/vllm-project/vllm/issues/54919

[V08] SM89 block FP8 dispatch: https://github.com/vllm-project/vllm/issues/58241

[V09] CUDA graph architecture source/design: https://github.com/vllm-project/vllm/blob/main/docs/design/cuda_graphs.md

[V10] FlashInfer verify graph improvement proposal: https://github.com/vllm-project/vllm/pull/60532

[V11] L4 native block FP8 CUTLASS proposal: https://github.com/vllm-project/vllm/pull/61224

[V12] Mamba continuation checkpoint proposal: https://github.com/vllm-project/vllm/pull/50551

[V13] NVFP4 KV experiment: https://github.com/vllm-project/vllm/pull/46963

[V14] Hybrid PP & MTP discussion: https://github.com/vllm-project/vllm/issues/60765

[V15] NCCL official environment reference: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

[V16] Batch-sharded sampling code discovery (versioned tree): https://github.com/vllm-project/vllm/tree/v0.32.0/vllm

[V17] vLLM release tag / source: https://github.com/vllm-project/vllm/tree/v0.32.0

[V18] PyTorch profiler (CUDA and NCCL critical-path tools): https://pytorch.org/docs/stable/profiler.html

[V19] FlashInfer repository: https://github.com/flashinfer-ai/flashinfer

[V20] Kaggle ARC-AGI-2 discussion supplied for context (speed claims **not reproduced**): https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/discussion/750546

[V21] NVIDIA NCCL performance/tuning: https://github.com/NVIDIA/nccl/blob/master/docs/userguide/source/troubleshooting/performance_and_tuning.rst

L. Update protocol — planned integration of external AI research

Version 1.0 establishes evidence level, baselines and an experiment-ID registry. For each newly supplied AI finding, add a row: **candidate / source URL / exact code or flag / expected mechanism / evidence hardware / known conflicting data / correctness risk / GPU test / result / next action**. Deduplicate identical proposals; if sources conflict, keep both with a resolution test. Future versions must preserve old baselines, log changed recommendations, and never silently convert hypotheses into measurements. **Current 1500+ status: target, not newly measured on our 4×L4 hardware.**

L. V2.0 সংযোজন: AI Research 01, দুটি পূর্ণ গবেষণার source-preserving integration

Intake: `Pasted text(9).txt`, 29 October 2026; প্রথম গবেষণার §1–14 এবং দ্বিতীয় গবেষণার §1–13 দুটোই পড়া হয়েছে। আগের dossier-এর বিষয় মুছে ফেলা হয়নি; এই সংযোজন add-on + change-of-priority। **এখন baseline পরিবর্তন অনুমোদিত**—কিন্তু engine/fork, checkpoint ও launch manifest পরিবর্তিত হলে তাকে নতুন arm হিসেবে ধরতে হবে। 1500+ sustained aggregate tok/s হলো user-set research target, verified NVIDIA reproduction নয়; এই দাবির প্রমাণ এখানে অনুসন্ধান করিনি।

L1. V2-এ অগ্রাধিকার বদলেছে কেন

1. **V-P0 (প্রথম):** SGLang INT4 quality gap এবং vLLM INT4-vs-FP8 model-quality effect বিচ্ছিন্ন করতে হবে; শুধু 1,024-token greedy boundary PASS দীর্ঘ ARC solve-quality certify করে না।
2. **V-P1 (খুব সস্তা):** `float16` GDN recurrent state → `bfloat16` A/B। Existing vLLM FP8 log-এ fused CUDA GDN spec decode-এর dtype eligibility নিয়ে সরাসরি warning আছে। Dtype change নির্ভুলতার নিশ্চয়তা নয়; kernel selection, memory, target-only logits পরীক্ষা করতে হবে।
3. **V-P1 (বড় structural opportunity):** pre-SM90 speculative **FULL CUDA graph** patch candidate #60532, padded full graph #53407, graph capture size issue #62556। Default 0.32 FlashInfer+MTP path PIECEWISE; শুধু `cudagraph_mode=FULL_AND_PIECEWISE` লিখলেই FULL verify হবে না।
4. **V-P1:** Hybrid prefix `--prefix-match-unit` এবং shared Mamba checkpoints; separate finalized decode checkpoint retention PR #50551। Tool-use turn-এর শেষে cached assistant output হারানো হয়েছে কি না পরীক্ষা।
5. **V-P1/P2:** MTP depth vs actual running concurrency, CUDA graphs, FP8 Marlin vs native Ada SM89 CUTLASS PR #61224, host-side FlashInfer plan caching এবং GDN split-K (Merlin patches)।
6. **V-P2/P3:** FP8 KV fixed sizing, KV scales, graph/memory trade-offs, TP1×4 with a different quantized checkpoint + host embeddings (Merlin), adaptive EAGLE-4, and PP4। High-risk/engineering track, not a new default.

L2. Comprehensive prioritized experiment registry (new findings preserved)

ID	Priority	পরিবর্তন / hypothesis	Cheapest decisive test	Reject / stop rule
V21-01	P0	INT4 target-only + FP8 reference; MTP on/off factorial	16 frozen prompts, first-token/logprob and 24–64 paired ARC tasks	Large-margin wrong state, malformed output, score regression
V21-02	P1	FP8 `--mamba-ssm-cache-dtype bfloat16` versus validated fp16	Only dtype delta, startup kernel selection, 8k/40k/90k throughput	No fused kernel, numerical regression, worse resident capacity
V21-03	P1	PR #60532 pre-SM90 speculative FULL graph source backport	Targeted source diff + small 8k verification replay & Nsight graph confirmation	Wrong backend/ABI, eager fallthrough, startup >600s, wrong outputs
V21-04	P1	Existing `--prefix-match-unit 16/32/64` + `--enable-mamba-shared-prefix-checkpoint`	3+ token-identical ARC agent turns; cached generated tail/TTFT	State mismatch, cache-hit not exercised, metadata OOM
V21-05	P1	CUDA graph capture-size grid for MTP verify/mixed batches (#62556)	Effective graph coverage and total resident memory vs 32/64/128/256/512/…	Graph OOM / reduced XL streams / no sustained benefit
V21-06	P1 gated	`disable_eagle_block_drop=true`	First record actual dropped block; repeated warm A/B with clean target oracle	Any high-margin divergence or stale recurrent state
V21-07	P1/P2	MTP3 vs MTP1 vs target-only at 8/12/16/20/24 active	One variable at a time, accepted/verify, draft cost, 90k admitted count	Bigger depth loses end-to-end or changes quality
V21-08	P1/P2	Merlin uniform multi-token graph / FlashInfer plan cache / second draft no-sync	Read source patch, off/on microprofile then A/B serve	Regression, unsupported flashinfer ABI, nondeterminism
V21-09	P2	Merlin GDN `in_proj_ba` split-K / RecoverSSM mechanisms	Isolated GEMM+GDN kernel benchmark with pinned dimensions	Incorrect GDN state or no whole-engine improvement
V21-10	P2/engineering	PR #61224 SM89 CUTLASS block-FP8 W8A8 path	Verify supported MMA shapes, kernel correctness, batch≥8, same FP8 weights	Significant register spills, batch1 slowdown dominant, wrong logits

ID	Priority	পরিবর্তন / hypothesis	Cheapest decisive test	Reject / stop rule
V21-11	P2	Fixed `--kv-cache-memory` near measured safe point	10.17→10.5→11.0GiB incremental, cold 90k, 12h-pattern bursts	OOM / less graph headroom / no extra actual running requests
V21-12	P2	Long-prefill scheduling, chunks 2048/4096/8192	Freeze resident decodes + inject 90k cold prefill; real agent replay	Decode starvation or excessive cold TTFT
V21-13	P2	NCCL CTA/channels, TP vocab gather and sampler sharding	1MB collective AND full model critical-path profile	No e2e benefit; unsupported shared topology
V21-14	P3	Merlin-style TP1×4 replicas, alternative ~16GiB INT4 checkpoint	One L4 model boot and 90k×1/2 memory gate first	One card OOM, fewer useful long streams, 10-min startup fail
V21-15	P3	TP1/PP4 vs TP4	Target-only short contexts, then MTP3 and shared-prefix XL	Pipeline bubbles, unsupported hybrid state, no genuine speed gain
V21-16	P3	EAGLE-4 hardware-adaptive drafter	Engine compatibility, 3B-class draft memory fit, kernel SM89, short acceptance	Unsupported old runtime, draft OOM, reasoning/tool regression
V21-17	P3	PR #50551 finalized Mamba decode checkpoints	Source diff then exact-turn warm hit + 50k run with cache-state trace	Correctness non-parity, checkpoint leak, unsupported code

Source grading: [LOCAL] existing 4×L4 reports; [UPSTREAM] other model/GPU/PR; [AUTHOR CLAIM] first-person Merlin/EAGLE report; [HYPOTHESIS] new 4×L4 untested branch. Upstream +36–47%, +42%, +62% বা +1.62× **এই L4 setup-এর forecast নয়**।

M. V-P1: FP16 → BF16 GDN state: one-flag, kernel-selection experiment

Attached research observation: validated vLLM 0.32 FP8 MTP3 + FP16 cache log says `Falling back to the Triton GDN spec decode path: the fused CUDA kernel requires ... BF16 or FP32 recurrent state`. This is a source-report of your log, not a new GPU run in this revision. The format BF16/FP16 both two bytes/element; however CPU/GPU kernels, scaling and state arithmetic differ, and available GDN-state slot headroom need not be identical.

Exact experimental delta (copy original full generated argv; change only this field):

```
# Baseline V21-02A
--mamba-ssm-cache-dtype float16
```

```
# New candidate V21-02B
--mamba-ssm-cache-dtype bfloat16
```

Validation sequence: (a) startup logs report fused CUDA or explicit reason for Triton fallback; (b) independent target-only BF16 oracle—not FP16 tokens—as numerical reference; (c) 8k/40k/90k resident decode at actual 8/12/16 live requests; (d) target MTP forced reject and prefix reuse, 2k/16k generated-token continuity; (e) 24–64 paired ARC tasks before adoption. Since the candidate changes numeric format, identical seed need not imply identical sampled output; look for high-margin and solve-rate regressions. **Fail:** kernel still Triton and speed not better, or CUDA selected but GDN logit/state diverges.

N. V-P1: FULL speculative verification graphs and capture-grid research

N1. #60532 pre-SM90 graph path (research supplied PR)

Research supplied: vLLM PR #60532, merged 22 Oct 2026, reportedly enables speculative verify FULL graph through FlashInfer FA2-compatible path on pre-SM90 GPUs. Author-reported Qwen3.8-27B on **SM80 NVFP4, TP1 73.6→108.1 (+47%), TP2 61→83.1 (+36%)**. These precise figures and merge status are **AI-supplied primary-source claims pending complete diff validation in this review**, not independently checked against 4×L4. PR: https://github.com/vllm-project/vllm/pull/60532

Implementation must not be one-line mode override: review backend graph-compatibility guard, `query_len` handling in speculative verify, attention backend choice, metadata/planning state and CUDA stream lifetime. Before any GPU: (1) pin original v0.32 source SHA; (2) fetch PR diff; (3) backport minimal compatible hunks; (4) build fully offline wheels; (5) verify model's actual graph dispatcher selects FULL verify; (6) compare eager/PIECEWISE/FULL target model next-token distributions. **Do not call a successful process launch a performance PASS.**

N2. #53407 padded uniform FULL graph and #62556 capture range

- PR #53407 adds padded uniform-decode FULL graph routing. **Not presumed present in pinned v0.32.0**, check with source. URL: https://github.com/vllm-project/vllm/pull/53407
- Issue #62556 has reproducible upstream graph-capture study: Qwen3.8-Flash-Next on 4×B300, expanded grid to 4096 improved reported c128 throughput 96.8%, c256 52.3%, with ~1.19GiB/GPU additional graph memory; by contrast **Qwen3.5-27B GDN H100 TP2 saw ~0 benefit and TP1 startup failed** due to too few Mamba blocks. This counterexample is more relevant than the high-percent headline. URL: https://github.com/vllm-project/vllm/issues/62556
- At MTP3 one decode/verify step can submit approximately `(draft_depth + bonus) × actual_running` query tokens, **plus any prefill work**. Record `effective_query_tokens`, captured size, graph-mode event, eager fallback fraction, graph VRAM, and Mamba slots. Start grid changes around **64/96/128/160/192/256**, not automatically 4096 on a 24GB GPU; exact allowed sizes from runtime config.

A/B: V21-05A current PIECEWISE, B FULL-capable backport with same graph sizes, C larger capture sizes after B passes. Re-run 90k capacity gate for every capture grid. `FULL_AND_PIECEWISE` requested without FULL verify selected = no graph optimization demonstrated.

O. Hybrid prefix-cache: partial match, shared junction, finalized continuation

O1. Upstream implementation lineage

- PR #46384 (merged July 2026, as reported) introduces partial prefix hits for Mamba+full-attention hybrid, lookups as fine as 16 tokens rather than physical 832/1648-token checkpoints, with partial Mamba state + copy-on-write. Compare logical hit length and restored state; physical block size not equal lookup unit. Source: https://github.com/vllm-project/vllm/pull/46384 ; documented reproducer: https://github.com/vllm-project/vllm/issues/51250
- Current v0.32 engine options include `--prefix-match-unit` and `--enable-mamba-shared-prefix-checkpoint`; shared checkpoint is valuable particularly when the same ARC prompt branches into 4 separate expert turns. Version-pinned docs: https://docs.vllm.ai/en/v0.32.0/features/automatic_prefix_caching/
- PR #50551 (reported as **under development**, exact source/merge requires checking) targets retention of finalized Mamba **decode** checkpoints, a distinct problem from prefill partial-hit matching. An upstream different-hybrid-model replay was reported to improve 63.6→94.7% cache hit; **not a Qwen3.8 L4 forecast**. Source: https://github.com/vllm-project/vllm/pull/50551
- Issue #53670 exposes speculative last-block drop and ~1648-token per-warm-turn recomputation. Its performance-ablation ≈206→322 and 149.6→241.9 tok/s is a warm workload observation, not pure decode speed. Source: https://github.com/vllm-project/vllm/issues/53670 ; related PR: https://github.com/vllm-project/vllm/pull/53388

O2. Four-stage controlled test; no blind release-switch

```
# V21-04A current flags exactly
# V21-04B add --prefix-match-unit 16
# V21-04C add --prefix-match-unit 16 --enable-mamba-shared-prefix-checkpoint
# V21-06D change only speculative JSON of the best passing C/B arm:
--speculative-config '{"method":"mtp","num_speculative_tokens":3,"disable_eagle_block_drop":true}'
```

Validate divisibility of the resolved cache-group physical block sizes, not an assumed number. Keep actual token-identical request branches: one prefill, 4 siblings, append 2k tool feedback, generate 2k–4k, repeat five times. Fields: prompt-token count, prefix-hit count, **cached generated continuation count**, prefill GPU time, warm TTFT, in-use Mamba blocks, actual resident requests, cumulative accepted output per wall second, first divergent token and tool validity. `disable_eagle_block_drop` stays **NO-GO for automatic production adoption**, due to unrelated upstream MTP+cache silent-corruption reports: https://github.com/vllm-project/vllm/issues/53912 .

Result classification: `NOT_EXERCISED` if no generated-tail reuse; `INCOMPLETE` if no valid cold control; `STATE_CORRUPTION` if linked stale restore + high-margin wrong next-token; `PERF_GAIN_ONLY` if faster but no task-quality validation; `CANDIDATE` only after paired ARC.

P. Merlin Engine: 4×TP1 research, memory layout, and five source-level patches

P1. Why this is new even though your TP2×2 failed

The publicly released Merlin Engine is a vLLM **0.32 fork**, not your installed official wheel. Its author-reported **4×CMP 170HX no-P2P, 8-agent 16k** aggregate output: TP4=371, TP2×2=523, TP1×4=873 tok/s. **Those CMP cards have 64 GB EACH**, not the L4's 24 GB; do **not** extrapolate that 873 number to 4×L4 or 90k independent prompts. Source: https://github.com/halt95/merlin-engine

Separate source: Merlin's different INT4 checkpoint **does claim 24GB single-card feasibility** (4-bit LM head and MTP, host-resident token embeddings, calibrated KV), unlike RedHat W4A16 with BF16 LM/MTP. It ships about 15.7GiB body + 210MiB draft and reports 271k KV pool on a **24GB RTX3090**. The card publishes single/dual/quad correctness checks, but not a L4 single-card proof. Source: https://huggingface.co/halt95/Qwen3.8-27B-W4A16-Merlin

New experiment V21-14: (1) check checkpoint license and quantized head numerical quality versus FP8; (2) one L4 single TP1 Merlin launch, runtime peak VRAM/host RAM and 90k+margin; (3) 2×TP1 then 4×TP1 with actual **per-replica capacity, task affinity and tool-agent routing**; (4) compare to one TP4 engine at equal total output budgets. Four single-card replicas which each fit **two 90k requests** provide only ≈8 such live sessions, not 20; if cache misses dominate, they may be slower despite eliminating NCCL. Cannot reuse current RedHat checkpoint and assume same fit.

Offline blocker: Merlin public build is reported with Python **3.13**, whereas Kaggle base is Python **3.12**. Test a fully offline Python3.13 interpreter/venv or rebuild matching Python3.12 wheels; verify CPython ABI, vLLM torch/cu13, sm89 compiled kernels, startup ≤600 s. Don't install a 3.13 wheel into 3.12.

P2. Actual code-level changes worth reading, not blindly copying

Merlin feature	Claimed mechanism	Our falsification benchmark	Integration limitation
Uniform multi-token CUDA graph	Full verify-step graph instead of dispatch overhead	effective verify graph coverage, critical-path latency	Version runner and FlashInfer API dependences
FlashInfer decode-plan cache	Skip repeated CPU-side attention planning	CPU metadata/sync flamegraph and 90k mixed batch	Key invalidation with changing prefix/context
Host-resident token embeddings	Recover VRAM for KV/Mamba state	Actual live 90k streams; PCIe embedding lookup cost	Current checkpoint layout differs
GDN `in_proj_ba` split-K	Faster fused GDN projection GEMM	tensor-shape microbench on SM89 + full model A/B	Correctness of fused quantized mapping
Second MTP draft step no-sync	Reduce CPU↔GPU blocking and launch gaps	Nsight CPU/GPU timeline with acceptance fixed	Device stream/data lifetime hazards
RecoverSSM / captured recurrent-state commit	Compact/replay speculative GDN state	forced reject + long reasoning + cached turn restore	High numerical/correctness risk

Sources: https://github.com/halt95/merlin-engine ; https://github.com/halt95/qwen38-27b-3090s ; https://github.com/halt95/qwen38-27b-3090s/releases . Before citing a patch as compatible, pin its exact commit/hunk. Merlin's P2P custom all-reduce is NOT portable to no-P2P Kaggle.

Q. FP8 SM89 native CUTLASS, KV and memory pool

Q1. PR #61224: native blockwise FP8 on Ada

AI research identifies vLLM PR #61224 (open at cutoff), reported author **1.42× whole-model Qwen3-8B-FP8 L4 throughput** (1,929→2,739 tok/s), batch32 20.16→15.11s, batch1 slightly worse; reportedly 226 kernel tests. This is Qwen3-8B, not 27B hybrid TP4 at 90k. The author describes a first slower kernel due to register spilling and optimized block-128 scaling; verify the exact pinned commit before port. Link: https://github.com/vllm-project/vllm/pull/61224 . Your v0.32 FP8 log selects `MarlinFP8ScaledMMLinearKernel`; native SM89 tensor cores alone do not mean the installed W8A8 block kernel dispatches.

A/B gate: dispatch to the new CUTLASS kernel only for validated supported shape sets, compare matmul FP32 reference error and graph compatibility, force same full model & FP8 weights, 8/40/90k at 8/16 actual requests. No quality degradation relative to fp8+Marlin; label source patch build separately. Older issue: https://github.com/vllm-project/vllm/issues/58241

Q2. KV memory and calibrated scales are not just capacity tuning

Saved vLLM FP8 `--mamba-ssm-cache-dtype float16` startup reported: GPU KV pool 1,123,048 tokens, usable memory 10.17GiB/GPU, CUDA graph actual 0.10GiB/estimated 0.74GiB. vLLM suggested **10,658,595,328 bytes** conservative KV allocation, **12,674,220,544** as full-utilization estimate (not safe). Testing 10.17 / 10.5 / 11.0 GiB is a **research proposal**, not an established 24GB-safe limit. Verify CUDA graph capture remains stable and Mamba slots/admission increase; no OOM for 90k prefill while 8–16 running.

```
# Use version-pinned actual flags and one candidate at a time, for example:
# --kv-cache-memory <measured_safe_bytes>
```

The FP8 KV warning `Using KV cache scaling factor 1.0 for fp8_e4m3` is not conclusive proof all per-layer scales equal 1.0. Compare checkpoint calibrated scales against runtime full-attention layer `k_scale_float` and `v_scale_float`, then greedy numerics BF16 KV control at shorter context. FP8 scales are an accuracy gate, not a pure speed knob.

Q3. 130k context is NOT covered by the original 106,496 launch

Current VMS vLLM FP8/INT4 throughput arms have `--max-model-len 106496`. To study 130k + generated continuation, launch **new** `max_model_len` with enough output reserve; record actual scheduler admission, GPU pool, startup and OOM. Do not represent XL ~90k result as measured 130k. E21 other 135,168-context arm is a different runtime/half; treat it separately.

R. Adaptive speculative decoding and EAGLE-4

The official EAGLE-4 Qwen3.8 drafter has an adaptive draft controller. Its **4×B300, TP4, concurrency 1, greedy** model card reports: vanilla 240.5, built-in MTP 534.5, DFlash2 672.6 and EAGLE-4 **863.8 tok/s**. EAGLE4/MTP speed ratio ≈1.62, EAGLE4/DFlash ≈1.28; T=1 also reported, but hardware/workload far from L4 and the output was limited to 4096. Sources: https://huggingface.co/EAGLE-Corp/Qwen3.8-27B-EAGLE4 ; https://github.com/eagle-hq/EAGLE-4 .

GPU-cost risk: dedicated BF16 drafter weights + KV/spec state may displace resident 90k streams. Published plugin uses patched/development SGLang/vLLM integrations; **not drop-in to stock vLLM 0.32**. Source/app license and competition redistribution terms require review; no assumption of Apache/MIT. If authorized and compatible, only after memory fit run 1/8/16/20 actual requests, acceptance, draft overhead, target verify graph, greedy forced rejection and paired ARC. Separate current built-in MTP head remains default until demonstrated better.

Adaptive depth without EAGLE-4: an external Intel XPU Qwen3.8-27B report says no-spec vs MTP1 vs MTP4 winners change strongly with concurrency: 4 streams 179/269/295, 16 streams 534/711/574, 32 streams 815/854/641, 64 streams 991/842/591. This is non-NVIDIA evidence for test selection, not an L4 prediction. Do not change MTP depth per request unless pinned engine natively supports dynamic routing. Proposed MTP3/MTP1/none arms by **actual running** 8/12/16/20/24. Source supplied: https://neural.download/ (full benchmark deep URL not furnished; **source location pending verification**).

S. Hardware feasibility, agent scheduling, and Kaggle adjacent results

S1. FP8 attention KV lower-bound; context matters

For TP1: `16 layers × 2(K,V) × 4 KV heads × 256-dim × 1 byte FP8 = 32768 bytes/token`. Ideal TP4 (4 KV heads evenly sharded): 8192 B/token/GPU. Ignoring GDN state, scale, fragmentation and speculative buffers:

Case	Theoretical attention KV only / GPU	Interpret
16×90k	11.8 GiB	Already a substantial fraction of L4 RAM
20×90k	14.7 GiB	Headroom further tight
20×130k	21.3 GiB	Leaves insufficient room for model + draft + runtime
24×130k	25.6 GiB	**Exceeds 24GiB physically** (unique full histories)

`20×90k` would require reading ~14.7GiB per GPU for an idealized full-context attention pass. Against a theoretical 300GB/s L4 bandwidth gives ~49ms as a very rough lower-bound if every byte must be read; FlashInfer tiling, prefix sharing, caching, attention reuse and speculative Q lengths complicate

this, so **not an exact decode latency prediction**. For reference: 90k VMS FP8 SSM fp16 currently 11 active, not 16; INT4 S20 12 active. Large-context problem is physical residency plus per-step computation, not just Python concurrency.

S2. Kaggle 4×L4 lessons (not Qwen3.8 apples-to-apples throughput)

- AIMO-2 fifth-place 14B AWQ reported three-engine 4×L4 throughput 304 LMDeploy / 263 SGLang / 238 vLLM (9 streams, ~13.5k context). Different model/version; useful to copy harness methodology (side-by-side environment, batch- and context-sweep).
- Another AIMO-2 7B AWQ batch32 SGLang ~1,103 tok/s was reported; **not transferable to 27B×90k**.
- Gemma 4 Developer Agent L4×4 notebook exposed CUDA graph warmup causing startup OOM despite modest LoRA weights; graph memory must be treated as resident-capacity cost, not just graph speed.
- ARC-AGI-3 FP8 repack discussions distinguish **faster checkpoint load** from model-quality improvement (no magically higher decode TPS).
- Konwinski, ARC Prize, AIMO-2 competitions provide offline wheel/driver/JIT and submission-timing lessons, but require exact public notebook links before claiming verified microbenchmarks.

Kaggle index: https://www.kaggle.com/competitions/ai-mathematical-olympiad-progress-prize-2 ; https://www.kaggle.com/competitions/gemma-4-developer-agent ; https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3 ; https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2 ; https://www.kaggle.com/code/abdullahmeda/load-72b-awq-model-using-vllm-on-l4-x4 . Exact AIMO 5th-place writeup direct link missing from pasted research; don't invent it.

S3. Mandatory three independent throughput definitions

1. **Resident decode:** accepted output tokens / time where all long prompts are prefilled; record effective running count and graph mode, not just request arrival.
2. **Mixed agent:** accepted output / total request wall including prefill, tool calls, model restarts, feedback and cache hit/miss; percentile and mean tail starvation.
3. **Submitted correct ARC work:** correct submitted scored units / scored run, coverage, output validity, solver time, 12h deadline drain. A 10% raw TPS gain with an ARC regression is an engineering **NO-GO**.

No 1500 claim until all three labeled; at minimum state **which definition** reaches 1500 and context/concurrency distribution. Upstream 4×B300 and high-VRAM CMP benchmarks never substitute for 4×L4 90k report.

T. Revision audit 01 — source coverage, conflicts, and evidence gaps

Audit protocol (run for every PDF update):

- **Pass A — completeness:** map every actionable insight in the incoming AI text to an experiment ID or an explicit exclusion; preserve numerical values, source hardware, caveat, and full raw URL.
- **Pass B — provenance:** source link actually resolves if possible; [LOCAL] reported, [UPSTREAM] verified card/issue, [AI-RESEARCH] provided but no original diff verified, [HYPOTHESIS] untested. If the incoming text only said “GitHub” without the underlying URI, record that missing source instead of fabricating a URL.
- **Pass C — feasibility:** actual v0.32 CLI support versus new PR fork/backport; sm89 and Python 3.12/cu130; no-P2P; 24GB memory; ≤600 s startup and 12h run.
- **Pass D — execution:** negative regression tests, frozen prompts/token IDs, target-only oracle, linked stale-state witness, no false PASS for skipped arm, artifact/report SHA, final coverage/quality.

AI Research 01 insight / claim	Coverage in this vLLM PDF	Audit disposition
Merlin 4×CMP TP1 873 vs TP2 523 vs TP4 371	P, V21-14	INCLUDED; 64GB CMP ≠ 24GB L4; 16k ≠ 90k
Merlin 16GiB checkpoint, INT4 lm_head+MTP, host embeddings	P, Q	INCLUDED; alternative weights change quality baseline
Merlin five code patches + RecoverSSM	P2	INCLUDED; individual A/B, not drop-in
EAGLE-4 863.8 and adaptive drafting	R	INCLUDED; B300, concurrency1, non-default integration
PR #60532 speculative FULL graph, reported +36–47%	N1	INCLUDED; complete PR diff requires direct verification
PR #53407, graph capture #62556 + counterexample	N2	INCLUDED; 24GB graph memory caveat
SM89 native FP8 PR #61224, 1.42× Qwen3-8B	Q1	INCLUDED; L4 author report, not Qwen3.8 evidence
BF16 recurrent fused GDN vs FP16 Triton fallback	M	INCLUDED; cheapest one-flag test
PR #46384 + #50551 partial/finalized Mamba caching	O	INCLUDED; independent mechanisms
MTP block-drop 206→322, corruption risks	O	INCLUDED; cold/warm strict gate
KV theoretical footprint, bandwidth, fixed-pool sizing	Q2, S1	INCLUDED; arithmetic assumptions explicit
MTP1/MTP3/no-spec high concurrency	R	INCLUDED; Intel XPU benchmark not L4
AIMO-2 4×L4 14B/7B, Gemma4 OOM and ARC-AGI-3 repack	S2	INCLUDED; direct AIMO source URL pending

AI Research 01 insight / claim	Coverage in this vLLM PDF	Audit disposition
PP4, TP1×4, no-P2P and Python wheel restriction	P, V21-14/15	INCLUDED; phased feasibility, no false prediction
Kaggle 900/980/710 and Merlin decode-only vs prefill distinction	S3	INCLUDED as **user-supplied/ unverified**; not needed for target proof
Flash-Next, TRT-LLM, ExLlama, LMDeploy surveys	S2 / scope note	OUT OF SCOPE as model/engine switch; deployment methodology retained

Important conflict resolution: An external report's initial SGLang INT4 PPL regression was later attributed by its author to stale CUDA driver cache. It is covered in the separate SGLang PDF and the cross-engine P0 comparison; do **not** label SGLang's quantized kernel permanently inaccurate based on the first issue post. Likewise, unrelated vLLM 0.19 RedHat garbage does **not** imply v0.32 INT4 is broken.

Additional raw URLs for Claude (complete spellings; no link-shortener)

- https://github.com/halt95/merlin-engine
- https://github.com/halt95/qwen38-27b-3090s
- https://huggingface.co/halt95/Qwen3.8-27B-W4A16-Merlin
- https://github.com/eagle-hq/EAGLE-4
- https://huggingface.co/EAGLE-Corp/Qwen3.8-27B-EAGLE4
- https://github.com/vllm-project/vllm/pull/60532
- https://github.com/vllm-project/vllm/pull/61224
- https://github.com/vllm-project/vllm/pull/53407
- https://github.com/vllm-project/vllm/issues/62556
- https://github.com/vllm-project/vllm/pull/50551
- https://github.com/vllm-project/vllm/pull/46384
- https://github.com/vllm-project/vllm/issues/53670
- https://github.com/vllm-project/vllm/issues/53912
- https://github.com/vllm-project/vllm/issues/58241
- https://github.com/vllm-project/vllm/issues/63593
- https://docs.vllm.ai/en/v0.32.0/features/automatic_prefix_caching/
- https://www.kaggle.com/competitions/ai-mathematical-olympiad-progress-prize-2

W. দ্বিতীয় AI-এর গবেষণা — নতুন source-backed experiments ও 1500+ অভিযানের অগ্রাধিকার

মূল নীতি (২৯ অক্টোবর ২০২৬): 1500+ sustained aggregate accepted-output tok/s আমাদের লক্ষ্য। অন্য GPU, engine fork, checkpoint বা quantization-এ প্রকাশিত benchmark থেকে **অ্যালগরিদম, source diff, kernel implementation, profiling পদ্ধতি ও পরীক্ষা তৈরির ধারণা** নিতে হবে। কোন পরিবর্তন Kaggle 4×L4-এ কাজ করবে, সেটি নতুন A/B দিয়ে নির্ধারণ করব। বর্তমান vLLM 0.32 configuration **স্থায়ী নয়**: TP/PP topology, quantization, MTP, KV/Mamba policy, CUDA graphs, checkpoint এবং selective source backport সবই পরিবর্তনযোগ্য—শর্ত হলো correctness, memory fit ও offline deployability পরীক্ষা।

ইনপুটের পরিধি: `Pasted text(10).txt`-এর মধ্যে **দুটি স্বতন্ত্র দীর্ঘ গবেষণা** ও শেষে পূর্ণ URL appendix আছে। নতুন অংশে তিনটিকেই অন্তর্ভুক্ত করা হয়েছে। আগের V1/V2-এর কোনো গুরুত্বপূর্ণ অধ্যায় বাদ দেওয়া হয়নি; নিচের priority table বর্তমান experiment ordering, আগের research ledger সংরক্ষিত থাকবে।

W1. vLLM 0.32-এর নতুন priority matrix

Priority	Experiment ID ও বিষয়	কেন প্রয়োজন	GO-এর প্রমাণ / STOP
P0	`V32-00` benchmark ও accuracy instrumentation	8k/40k/90k/130k-এ real running count এবং clean reference	Missing evidence/incorrect output হলে fail-closed
P0	`V32-01` MTP local argmax	TP4 vocabulary all-gather কমানোর existing-source option	Greedy draft IDs অক্ষত; NCCL time ও TPS উন্নতি
P0	`V32-02` Mamba fine-grained prefix matching	First-repeat miss, বড় alignment এবং generated tail re-prefill	তিন turn-এ cached-token improvement; state parity
P0	`V32-03` mixed-prefill/decode starvation	Cold 90k prefill চলাকালে resident decode আটকে যাচ্ছে কি না	19+1/16+4 workload-এ wall TPS ও latency
P1	`V32-04` PR #60532 FULL graph backport	FA2 speculative verify-এর graph coverage	বাস্তব FULL replay, no corrupted output, KV fit
P1	`V32-05` NCCL TP4 microbench+trace	4×L4 PCIe SHM collective latency	Actual critical path কমে, engine TPS বাড়ে
P1	`V32-06` finalized checkpoint ও MTP block retention	Multi-turn assistant suffix আবার prefill হওয়ার ক্ষতি	Generated suffix reuse, no stale checkpoint
P1	`V32-07` SM89-native FP8 GEMM #61224	Current Marlin block-FP8 path-এর বিকল্প CUDA kernel	GEMM/output parity এবং 40k/90k gain
P2	`V32-08` PP4 / TP1 এবং replica layout	TP4 communication কমানোর নতুন architecture	Model fit, no GDN corruption, 90k benchmark
P2	`V32-09` concurrency-aware MTP depth	8/12/16/20 batch-এ MTP3 সবসময় optimal কি না	Same-output-budget accepted TPS বৃদ্ধি

Priority-এর অর্থ: P0 হচ্ছে প্রথমে করার পরীক্ষা, production adoption নয়। Profiler যদি দেখায় verification graph fallback-ই প্রধান খরচ, `V32-04`-কে আরও আগে নেওয়া যায়। একসঙ্গে তিনটি knob বদলে gain attribution নষ্ট করা যাবে না।

W2. V32-01 — MTP local-argmax reduction: TP4 all-gather কমানোর সস্তা পরীক্ষা

vLLM **0.32.0**-এর `SpeculativeConfig.use_local_argmax_reduction` option নথিভুক্ত আছে। Greedy, non-tree speculative draft-token নির্বাচন করতে পুরো vocabulary logits gather না করে প্রতিটি TP rank-এর local maximum থেকে global winner বের করে। Source অনুযায়ী draft-selection যোগাযোগের জটিলতা প্রায় **O(V) → O(2×TP)**। এটি **target model-এর সব all-reduce বন্ধ করে না** এবং non-greedy sampling-এর universal optimization নয়।

শুধু speculative JSON পরিবর্তন করে একটি arm:

```
{"method":"mtp", "num_speculative_tokens":3, "use_local_argmax_reduction":true}
```

পরীক্ষা: MTP3 baseline বনাম এই arm, 8k/40k/90k, সত্যিকার admitted 8/12/16/20 streams। প্রতিটি greedy draft token ID, full target output, logit margin, accepted length, `all_gather` সময়, sampler সময়, target verify, end-to-end tok/s record করুন। Enable করার পরও implementation option-টি উপেক্ষা করলে সেটি `NOT_EXERCISED`, PASS নয়। Unsupported proposer, high-margin token mismatch বা tool-output corruption হলে STOP। `use_heterogeneous_vocab`-এর সঙ্গে issue #51561-এ আলাদা interaction report আছে; যাচাই ছাড়া একসঙ্গে enable নয়।

Source URLs:

https://docs.vllm.ai/en/v0.32.0/api/vllm/config/

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/config/speculative.py

https://github.com/vllm-project/vllm/issues/51561

W3. V32-02/06 — Prefix-cache failure-এর তিনটি আলাদা কারণ ও তিন-turn ladder

একই agent conversation-এ cache loss-এর অন্তত তিনটি ভিন্ন mechanism থাকতে পারে। **(ক)** প্রথম repeated request-এ structurally matched KV থাকলেও aligned Mamba checkpoint নেই, **(খ)** MTP/EAGLE শেষ aligned block ফেলে দেওয়ায় একাধিক হাজার tokens পুনরায় prefill হয়, **(গ)** আগের assistant-generated tokens-এর finalized checkpoint সংরক্ষিত হয় না। তিনটির fix এক নয়; মিলিয়ে পরীক্ষা না করলে প্রাপ্ত improvement-ও ভুলভাবে যোগ করা হবে।

পাঁচটি স্বতন্ত্র arms:

Arm	পরিবর্তন	মাপা হবে
B0	Current `align`, MTP3, cache ON	Baseline first-repeat/second-repeat hit
B1	`--prefix-match-unit 64` (বৈধ হলে)	Physical-block-এর ভেতরে finer prefix hit
B2	B1 + `--enable-mamba-shared-prefix-checkpoint`	Branch junction-এ valid Mamba checkpoint
B3	B2 + version-checked finalized checkpoint PR #50551	Generated assistant tail পুনর্ব্যবহার
B4	`disable_eagle_block_drop=true` **একেবারে আলাদা high-risk arm**	Protection বাদ দিয়ে retained block-এর gain/risk

64 যদি সব cache group-এর resolved block size-কে divide না করে, **32 বা 16** আলাদা startup-validated arm হিসেবে পরীক্ষা করুন। 8k/40k/90k prefix-সহ একই request তিনবার পাঠান; তারপর last 1/16/64/512 tokens বদলে branch পরীক্ষা। Log: `cached_tokens`, `cached_generated_tokens`, re-prefilled tokens, prefix LCM, checkpoint position, cold-vs-warm full-logit/greedy output এবং actual wall-time savings। Upstream #53670-এ বড় alignment block পুনরায় prefill হওয়ার কারণে warm throughput কমার বাস্তব উদাহরণ আছে; আমরা তার **trigger/repair methodology** নেব। B4-এ stale speculative state বা output drift হলে তৎক্ষণাৎ rollback।

Primary URLs:

https://github.com/vllm-project/vllm/issues/53504

https://github.com/vllm-project/vllm/issues/53670

https://github.com/vllm-project/vllm/pull/46384

https://github.com/vllm-project/vllm/pull/50551

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/config/cache.py

https://docs.vllm.ai/en/v0.32.0/features/automatic_prefix_caching/

W4. `V32-03` — SM89 long-prefill/decode starvation: পাঁচ ধরনের controlled workload

vLLM issue #54919-এ Qwen3.8 hybrid model-এর SM89 reproduction-এ **KV pool শেষ না হলেও** দুইটি বড় uncached prefill চলাকালে active decode throughput প্রায় থেমে গিয়েছিল। Cached-prefix control স্বাভাবিক ছিল; MTP on/off উভয় অবস্থাতেই সমস্যা দেখা যায়। তাই এটিকে শুধু VRAM shortage ধরে tuning করলে ভুল হবে। Attention-metadata host synchronization একটি profiling lead; একমাত্র নিশ্চিত root cause বলে ধরে library monkeypatch করা ঠিক নয়।

Test	Traffic	কোন ফল গুরুত্বপূর্ণ
M0	20 resident decoders, নতুন prefill নেই	True stable decode ceiling
M1	19 resident + 1 **warm** 90k	Prefix-hit scheduler overhead
M2	19 resident + 1 **cold** 90k	Decode starvation-এর স্থায়িত্ব
M3	16 resident + 4 cold 90k	Interference এবং queueing scale
M4	Real tool-call timeline-এ staggered arrival	Whole-agent accepted tok/s ও solved/hour

M2–M4-তে `--max-num-batched-tokens` 2048/4096/8192 আলাদাভাবে sweep করুন, scheduler-এর effective admission ও `long_prefill_token_threshold` record করুন। Requested 20 বাস্তবে resident না হলে actual running count দিয়ে মাপুন। Full interval-এর accepted output tokens/wall-time এবং steady decode seconds—দুটোই দিন। Smaller chunks decode fairness বাড়ালেও cold prefill ধীর করতে পারে। GPU profiler trace, Python/API stalls এবং TTFT p95 দিয়ে trade-off বুঝুন। `.cpu()` synchronization blind remove করা যাবে না—নতুন async kernels-এ correctness সমস্যা পাওয়া গেছে।

https://github.com/vllm-project/vllm/issues/54919

https://github.com/vllm-project/vllm/issues/61239

W5. V32-04 — CUDA graphs: পুরোনো ভুল hypothesis বাদ, নতুন verify FULL path পরীক্ষা

Source-correction গুরুত্বপূর্ণ: issue #51008-এর প্রথম diagnosis ছিল, MTP proposer নাকি প্রতি step-এ 48-layer GDN eager forward চালায়। লেখক পরে per-step profiler দিয়ে **নিজের ওই দাবি খণ্ডন করেছেন**। Uniform target decoding-এর GDN FULL graph-এ captured ছিল; MTP `EagleProposer` target hidden states ব্যবহার করে, 48-layer model পুনরায় চালায় না। ফলে "proposer-কে আবার graph capture করলেই 48-layer eager overhead যাবে"—এমন patch-এ quota নষ্ট করা যাবে না। Mixed/nonuniform prefill অবশ্যই আলাদা profiling target।

PR **#60532**-তে pre-SM90 FlashInfer FA2 speculative verification-কে FULL graph-eligible path-এ নেওয়ার code change আছে; v0.32.0-এর original release-এর পরে হয়েছে। Source diff থেকে প্রয়োজনীয় backend routing, query length ও graph declarations **একসঙ্গে** backport করে sm_89/cu13-compatible offline wheel বানাতে হবে। শুধু `--compilation-config` দিয়ে FULL request করলেই actual FULL replay হবে না। Measure করুন: effective verify graph mode, per-step graph hit/fallback, MTP acceptance, CUDA graph VRAM, KV capacity, cold startup/JIT, same-token parity, 8k/40k/90k speed। Graph memory বেশি খেয়ে XL resident count কমালে reject। PR #53407-এর padded uniform graph এবং issue #62556-এর capture-size/capacity trade-off-ও একই experiment family-তে রাখুন।

https://github.com/vllm-project/vllm/issues/51008

https://github.com/vllm-project/vllm/pull/60532

https://github.com/vllm-project/vllm/pull/53407

https://github.com/vllm-project/vllm/issues/62556

https://github.com/vllm-project/vllm/blob/main/docs/design/cuda_graphs.md

W6. V32-05/07 — NCCL SHM এবং native Ada FP8 source-পোর্ট

NCCL: `nccl-tests`-এ আপনার actual TP4 collective message sizes, host SHM, capture mode ও channel count রেখে baseline তৈরি করুন। তারপর CTA 1/2/4/8, Ring/Tree, Simple/LL, এবং installed NCCL-এ সমর্থিত হলে `NCCL_SHM_USE_CUDA_MEMCPY`—প্রতিটি আলাদা arm। `NCCL_DEBUG=INFO` ও profiler দিয়ে সত্যিই algorithm/transport বদলেছে কি না যাচাই করুন। Standalone microbenchmark দ্রুত হলেও GDN kernels-এর SM-time কেড়ে নিয়ে engine step ধীর হতে পারে; final judge হবে output/wall tok/s ও non-overlapped critical path। আগের PCIe-IPC custom-allreduce regression একটি **negative control**; আলাদা staged communication algorithm পরীক্ষা করার নিষেধ নয়।

SM89-native FP8: PR #61224-এ CUTLASS block-FP8-এর CUDA implementation ও L4 benchmark আছে। এখান থেকে tile/epilogue/packing strategy ও kernel tests নিন। Actual vLLM 0.32-এর `MarlinFP8ScaledMMLinearKernel`-এর সঙ্গে matched GEMM shapes-এ numerical A/A, 27B first-logit parity, তারপর 8k/40k/90k engine speed। Torch/CUDA version ও offline cubin compilation compatibility না পেলে production adoption নয়। PR #58241-এর capability gating দেখুন।

https://github.com/NVIDIA/nccl-tests

https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

https://github.com/vllm-project/vllm/issues/58241

https://github.com/vllm-project/vllm/pull/61224

W7. `V32-08/09` — PP4, memory layout, concurrency-dependent MTP

TP4 → PP4/TP1: PCIe collective overhead এড়াতে pipeline stages বা independent replicas গবেষণাযোগ্য। অন্য setup-এর Merlin results থেকে **source code, single-card quantization, embedding placement, scheduler এবং memory accounting** উদ্ধার করুন। PP4-এ GDN recurrent state, target/MTP head placement, interstage communication, cache-group shape এবং 24GB stage balance কঠিন; issue #63505-এ vLLM 0.32 PP hybrid failure-এর একটি concrete negative test আছে। তাই ধাপে ধাপে `target-only short → 2k prefill → 40k → 90k cached → MTP3 → 16+ streams`; কোনো ধাপে garbage/illegal-memory হলে STOP, architecture বাতিল ঘোষণা নয়।

Adaptive MTP: অন্য engine-এ high concurrency-তে shallow/no speculation-এর throughput ভালো হওয়ার উদাহরণ আছে। **লাভের অ্যালগরিদম** হলো real accepted tokens per verify / verify cost অনুযায়ী depth নির্বাচন। প্রথমে MTP OFF, K1, K3 কে 8/12/16/20 actual streams-এ স্থির arms হিসেবে মাপুন; v0.32 native dynamic switching যাচাই ছাড়া Python client থেকে spec config বদলাবেন না।

Capacity: FP8 full-attention KV TP4-এর আনুমানিক ব্যয় 8192 bytes/token/GPU। 24×130k independent histories-এর attention KV-ই প্রায় 25.6GB/GPU, কিন্তু shared-prefix cache ও queueing ব্যবহার করে বেশি **logical concurrent agents** চালানো যায়। এটিকে research design-এর input হিসেবে ধরুন; 1500+ লক্ষ্যকে খারিজ করার কারণ নয়। `resident decode peak`, `mixed-agent sustained`, `correct submitted outputs per hour`—তিনটি লক্ষ্য আলাদা রিপোর্ট করুন।

https://github.com/vllm-project/vllm/issues/63505

https://github.com/vllm-project/vllm/issues/63593

https://github.com/halt95/merlin-engine

W8. DFlash cache corruption-এর সম্পূর্ণ follow-up থেকে শেখা

Issue #60174-এর প্রথম report-এ **DFlash2/DSpark + prefix caching + sliding-window drafter**-এর কয়েক turn পর output loop হয়েছিল। Follow-up debug-এ draft sliding-window block, primary GDN block alignment, padded LCM এবং attention backend গুরুত্বপূর্ণ পাওয়া যায়; নির্দিষ্ট older build-এ drafter-কে `TRITON_ATTN`-এ নেওয়ার workaround কাজ করেছে। **গুরুত্বপূর্ণ শেষ update:** original reporter-এর **vLLM 0.32.0 re-test পরিষ্কার** হয়েছে—পুরোনো failure-কে স্থায়ী 0.32 bug বলা যাবে না।

আমাদের takeaway হলো **test implementation**: তিন থেকে পাঁচটি growing-prefix chat turns, প্রথম valid hit, draft-group block size/LCM, first-corrupt token, MTP acceptance এবং cache-hit logs সংরক্ষণ; 0.32-এ নতুন alternative drafter/graph change দিলে আবার এই negative test চালাতে হবে। #63593-এর K≥5 reject-state bug থেকে K3-এর verdict অনুমান করা যাবে না। Correctness শুধু short `2+2` smoke দিয়ে certify নয়।

https://github.com/vllm-project/vllm/issues/60174

https://github.com/vllm-project/vllm/pull/55601

https://github.com/vllm-project/vllm/pull/60252

https://github.com/vllm-project/vllm/issues/63593

X. Version 3 — দ্বিতীয় AI গবেষণার পূর্ণ coverage audit

এই audit-এ incoming লেখার **দুটি মূল গবেষণা**, final URL appendix এবং আগের V1/V2 sections আলাদাভাবে মিলিয়েছি। নিম্নের কোনো insight কেবল summary-তে আটকে নেই—সংশ্লিষ্ট experiment ও test procedure-এ বিস্তারিত আছে।

দ্বিতীয় AI-এর গুরুত্বপূর্ণ finding	অবস্থান	Audit result
SM89 cold-prefill starvation, M0–M4 ও chunk sweep	W4	**INCLUDED**
First-repeat / 1600–3100 reprefill / finalized Mamba checkpoint	W3	**INCLUDED**, তিন mechanism পৃথক
MTP greedy local argmax O(V) → O(2TP)	W2	**INCLUDED**, exact JSON ও verification
MTP eager-GDN claim পরবর্তী profiling-এ ভুল প্রমাণিত	W5	**CORRECTED**, ভুল optimization বাদ
#60532 FULL-verify FA2, #53407 এবং graph-size	W5	**INCLUDED**, source-backport steps
#61224 SM89 FP8, #58241 Marlin fallback	W6	**INCLUDED**, kernel parity
NCCL Ring/Tree/CTA/SHM memcpy/ graph	W6	**INCLUDED**, microbench vs engine
PP3 issue #63505 → PP4 staged testing	W7	**INCLUDED**, negative controls
MTP depth adaptive / no-spec high concurrency	W7	**INCLUDED**, fixed-arm sweep
#60174 old corruption + v0.32 final resolution	W8	**INCLUDED**, full issue chronology
#63593 K≥5, আমাদের K3 tested separately	W8	**INCLUDED**, no false conclusion
24×130k KV lower bound / shared-prefix scheduling	W7	**INCLUDED**, engineering adaptation
Flash-Next ও অন্য engine-এর transferable techniques	আগের V2 P/S sections	**PRESERVED**, model switch out of scope
AIMO2/Gemma4 cross-competition experiments	আগের S2/primary source sections	**PRESERVED**, idea extraction
Source-এর 40+ raw URLs	W2–W8 + আগের source ledger	**PRESERVED/ADDED**, clickable

পরবর্তী revision-এর বাধ্যতামূলক দুই audit: (১) আগের Markdown/PDF-এর সব headings, research claims ও source URL রাখা হয়েছে কি না; নতুন AI-এর প্রতিটি actionable insight নতুন experiment-এ map হয়েছে কি না। (২) PDF-এর বাংলা glyph, table overflow, page boundary, clickable URL, experiment ID, benchmark values এবং source provenance ঠিক আছে কি না। কোনো পূর্ণাঙ্গ গবেষণাকে ছোট summary দিয়ে replace করা যাবে না।

Y. Version 4 — তৃতীয় AI-এর সম্পূর্ণ গবেষণা সমন্বয়: vLLM 0.32.0

নতুন গবেষণা উৎস: `Pasted text(20261029-065936).txt`-এর দুটি দীর্ঘ গবেষণা অংশ (প্রথম: cache/agent scheduling/Duck; দ্বিতীয়: local-argmax/NCCL/Mamba retention/Kaggle L4)। আগের v1/v2/v3-এর সব বিষয় রাখা হয়েছে। এই অধ্যায় নতুন mechanism, যাচাই ও experiment priority যোগ করে; পুরোনো baseline স্থায়ী করে না। **আমাদের লক্ষ্য 1500+ sustained aggregate accepted-output tok/s**, গবেষণাপত্রে 700–1000 থাকলেও সেটিকে আগের মধ্যবর্তী milestone গণ্য করা হবে।

Y1. Updated experiment order — vLLM-এর প্রথম ১৫টি নির্দিষ্ট পরীক্ষা

Rank	ID	কাজ / প্রথম independent variable	প্রত্যক্ষ মূল্য	Gate / Stop
P0	V4-00	পুরোনো agent event-log replay: real throughput তিন ভাগ	idle/prefill/decode bottleneck নির্ণয়; GPU শূন্য	missing timestamps হলে INCOMPLETE
P0	V4-01	MTP3 `use_local_argmax_reduction=true`	TP4 vocabulary logit all-gather সরানোর সম্ভাবনা	greedy global argmax/tie parity
P0	V4-02	`prefix_match_unit` default → 64 → 16	hybrid cache granularity; first-repeat miss	cold/warm/GDN-state match
P0	V4-03	best prefix-match arm + shared-prefix checkpoint	tool-return junction checkpoint	old/new states, no leak
P0	V4-04	decode-only vs +1/+4 cold 90k prefill	prefill-induced starvation isolate	equal total work, TPOT+p95
P1	V4-05	`disable_eagle_block_drop` ONLY on its own arm	last MTP cache block reuse	multi-turn forced-reject clean
P1	V4-06	application-directed checkpoint/eviction policy	preserve valuable branch points	exact source/backport provenance
P1	V4-07	short cache-aware agent session lease	retain cache over brief tool calls	fairness, exact work budget
P1	V4-08	12/16/18/20/22/24 actual running	highest useful occupancy at 8k/40k/90k	no OOM/retractions
P1	V4-09	FULL verification CUDA graph PR #60532	target verify critical-path improvement	corrected graph ON evidence
P2	V4-10	SM89 block FP8/ CUTLASS PR #61224	FP8 GEMM kernel; source-level port	kernel parity, offline binary
P2	V4-11	low-M GEMM/kernel dispatch benchmark	target-verify batch crossover	actual CUDA time beats baseline

Rank	ID	কাজ / প্রথম independent variable	প্রত্যক্ষ মূল্য	Gate / Stop
P2	V4-12	fixed KV vs speculative graph-memory cap	effective resident streams	min headroom, no OOM
P2	V4-13	NCCL SHM small collective + planner sync	reduce TP4 overhead	profiler critical path reduction
P3	V4-14	fork-derived graph/ plan-cache/GDN patches	extract Merlin / Franzen ideas individually	isolated compile+accuracy gate

Scheduling rule: v3-এর experiment ledger-এর সব আইডি এখনও valid। এই table নতুন work-order; একই optimisation (যেমন prefix matching) দুবার চালানো নয়। প্রতি arm-এ exact checkpoint, server source SHA, resolved config, test inputs, accepted tokens, actual admitted N, output validity এবং startup seconds সংরক্ষণ করতে হবে। 8k-এ gain কিন্তু 90k-এ loss হলে সেটিকে **context-specific** result বলুন, generic improvement নয়।

Y2. vLLM hybrid prefix cache: first-repeat miss, alignment ও trailing block — তিনটি পৃথক problem

[UPSTREAM] #53504-এ Qwen3.8-27B FP8 + MTP-এর first repeated request cache-miss lead; #52244-এ hash unit, one-token cap এবং aligned Mamba checkpoint একসঙ্গে cache lookup নষ্ট করতে পারে বলে গবেষণায় দাবি; #53670-এ speculative last-block eviction প্রতি warm turn-এ অতিরিক্ত 1,500–3,100 tokens re-prefill-এর evidence। #52244-এর নির্দিষ্ট PR mechanism এই revision-এ সম্পূর্ণ source-validated নয়: এটি source-to-check, merged behavior ধরে নয়।

কেন আলাদা arm চাই: (i) hash granularity কমালেই Mamba checkpoint থাকবে এমন নয়; (ii) junction checkpoint থাকলেও final speculative block dropped হতে পারে; (iii) final block রাখা numerically safe কি না, সেটি independent। তাই v4-এর V4-02, V4-03, V4-05 তিনটি পৃথক A/B।

```
# V4-02a: only match unit; server should validate physical block divisibility.
--prefix-match-unit 64

# V4-02b: independent 16-token unit candidate, if every group supports it.
--prefix-match-unit 16

# V4-03: best validated unit plus checkpoint; other flags unchanged.
--prefix-match-unit 16 \
--enable-mamba-shared-prefix-checkpoint

# V4-05: HIGH-RISK separate arm, never automatically paired with V4-03.
--speculative-config '{"method":"mtp","num_speculative_tokens":3,"disable_eagle_block_drop":true}'
```

৫-রাউন্ড multi-turn test: একই 50k common prompt → 4–20k thinking → tool call → real tool result → new continuation → 90k–130k; প্রতিটি round-এ exact token IDs, `cached_tokens`, generated-thinking portion cached, aligned Mamba boundary, `uncached_prompt_tokens`, prefill seconds, actual solver output ও last-block retained/evicted log। `first_repeat`, `second_repeat` ও `branch_point` আলাদা। `--disable-eagle_block_drop`-এর পূর্বে target-only cold oracle এবং force-rejected MTP3 state parity বাধ্যতামূলক।

Source URLs

https://github.com/vllm-project/vllm/issues/53504

https://github.com/vllm-project/vllm/pull/52244

https://github.com/vllm-project/vllm/issues/53670

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/config/cache.py

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/config/speculative.py

Y3. Cache-retreat: FA blocks নয়, matching GDN checkpoint-ই আসল resumability নির্ধারণ করে

[UPSTREAM, DIRECT PRIMARY SOURCE] vLLM `#57111`-এর checkpoint-aware eviction + segmented recomputation RFC-তে দেখানো হয়েছে, completed request-এর full-attention block eviction এবং recurrent state checkpoint eviction ভিন্ন দিকে ঘটতে পারে। অল্প কিছু FA block হারিয়েও expensive middle-context recompute লাগতে পারে; RFC-র proposed repair interval `[repair_start,repair_end)` এর পরের resident suffix reuse করার design **কোনো current v0.32 built-in guarantee নয়**।

[AI-RESEARCH LEAD] `#55697` application-directed Mamba checkpoint retention: shared prefix + tool-return/resume boundary-র checkpoint বেশি মূল্যবান; reported 2.1× QPS ছিল ভিন্ন short application-এর, আমাদের ARC projection নয়। `#40865` SGLang-এ 93% checkpoint retreat ও ~30% prefill compute: mechanism V4 cache-research-এ transfer করে পরীক্ষা করতে হবে, copy-paste implementation নয়।

Low-risk measurement first: per request `FA_prefix_hit`, `Mamba_compatible_checkpoint_depth`, `retreated_tokens=FA_hit - effective_hit`, `checkpoint_evicted`, `tool_return_delay_ms` এবং `re_prefill_tokens` রিপোর্ট। **High-value negative test:** একই 60k prompt-এর last 1/16/64/512 tokens বদলানো এবং tool result append; current checkpoint eviction pressure কম/বেশি রেখে repeat। যদি extra checkpoint pool KV capacity কমিয়ে 90k concurrency নষ্ট করে, gain reject।

Source URLs

https://github.com/vllm-project/vllm/issues/57111

https://github.com/vllm-project/vllm/issues/55697

https://github.com/sgl-project/sglang/issues/40865

https://github.com/vllm-project/vllm/pull/46384

Y4. `use_local_argmax_reduction`: ছোট flag হলেও source-level high-value TP4 experiment

[UPSTREAM/EXACT-VERSION] vLLM 0.32.0 speculative config-এ greedy non-tree draft selection-এর জন্য rank-local argmax-এর result reduce করার option আছে। বড় vocabulary logits সব rank থেকে all-gather করার বদলে অল্প scalar information পাঠানো হয়। এই mechanism correctness-conserving intention—numerical ties, per-rank vocabulary offsets এবং relevant MTP mode পরীক্ষা ছাড়া tested-safe নয়।

```
--speculative-config
'{"method":"mtp","num_speculative_tokens":3,"use_local_argmax_reduction":true}'
```

A/B: only this flag; 1/8/16/20 running, 8k/40k/90k; target-selected token IDs, rank-local score offsets, accepting step, number of vocabulary all-gathers ও NCCL kernel time। **Negative injection:** global winning index TP rank 0/1/2/3-এর প্রত্যেকটিতে, negative logits, exact tie এবং invalid rank offset; Python independent argmax oracle-এর সঙ্গে মিলিয়ে test। Any actual argmax mismatch ⇒ NO-GO। Notebook config JSON-এর field support source থেকে startup-এ assert করবেন; silently ignored flag নয়।

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/config/speculative.py

Y5. 90k throughput কমার কারণ জানতে চারটি controlled prefill/decode workload

vLLM #54919-এ SM89 Qwen3.8-family long cold prefill চলাকালে decode starvation report রয়েছে; MTP ON/OFF দুই ক্ষেত্রেই। ফলে prefill scheduler আর kernel-side full-attention/FlashInfer host-sync আলাদা করে profile করতে হবে। অন্য issue #51008-এর আলোচনায় "MTP proposer প্রতি uniform decode-এ eager GDN" hypothesis পরের instrumentation-এ ভুল প্রমাণিত: সেই ভুলের ওপর graph-off rewrite করবেন না।

Probe	Exactly controlled work	Report
M0	20 warm resident decode, no new prefill	saturated decode baseline
M1	19 warm + one 90k cold	prefill interference
M2	16 warm + four 90k cold	worst mixed batch
M3	20 staggered, realistic tool-return arrivals	real agent wall TPS

M0–M3 প্রত্যেকটি `--max-num-batched-tokens 2048/4096/8192` পৃথক server runs; same total target-generated tokens এবং fixed logical request IDs। Measurement: p50/p95 decode step, prefill overlap, unfinished prefills, CUDA graph hit rate, actual active streams, wall-correctly-emitted tok/s, tool-time, first-token latency। Smaller prefill chunks decode starvation কমাতে পারে কিন্তু total prefill time বাড়াতে পারে: gain দুই metric-এ বিচার করুন।

https://github.com/vllm-project/vllm/issues/54919

https://github.com/vllm-project/vllm/issues/51008

Y6. Low-batch kernels, Merlin/Franzen, GPU sampling — optimizations যেখান থেকে actual implementation আহরণ করব

- **[UPSTREAM]** single-L4 llama.cpp Qwen3.8/DFlash2 benchmark-এ verification GEMM dispatch boundary বদলে ~16.2% faster হয়েছিল। একই kernel vLLM Marlin-এ চলবে—এমন দাবি নয়; **testable principle:** target verification-এর effective matrix M 1–8 ও 8–24-এ কোন GEMM/quant dispatch নির্বাচিত হচ্ছে? SM89 target-specific kernel selection profile।
- **[UPSTREAM]** Franzen Duck/SGLang fork-এ low-M GEMM, strategically retained Mamba prefill checkpoints, LRU refresh, speculative state memory budget এবং larger history-trimming blocks; source code থেকে minimal logic আলাদা, TP4/SM89 adaptation test। তাঁর low-M BF16 patch SM120 implementation হলে L4-compatible substitute develop করতে হবে; direct binary reuse নয়।
- **[RESEARCH]** Merlin vLLM fork-এর graph capture, FlashInfer plan cache, host-resident embeddings, split-K GDN and no-sync proposal একেকটি patch আলাদা benchmark করুন। Current

scope-এ model Qwen3.8-27B ও vLLM 0.32.0-ভিত্তিক fork থাকবে; checkpoint quantization/model class বদলানো গেলে independent oracle requalify করুন।

- **TP vocabulary/output-head:** 248k vocabulary sampling/gather ও host transfer profile; sampling semantics না বদলে communication কমানোর অপশন আগে।

Exact per-kernel trace: `M (verify query rows)`, `N,K`, quant dtype/pack, `kernel_name`, `cuda_time_ms`, `graph_replay_mode`, number TP allreduces/gathers, `accepted_outputs_per_verify`। শুধু 1-token throughput নয়; representative verify widths 2/4/6/8 এবং concurrency 8/16/20।

https://github.com/hanxiao/Qwen3.8-27B-UD-Q4_K_XL-L4

https://github.com/da-fr/arc-agi-3-solution

https://github.com/da-fr/arc-agi-3-solution/blob/main/WRITEUP.md

Y7. Agent-level cache-aware session lease — zero-kernel research arm

Mechanism: short Python tool call শেষে assistant-এর same active task দ্রুত resume করলে existing KV/Mamba state retain হওয়ার সুযোগ বাড়ে; অন্য 100k cold task ঢোকাতে গিয়ে valuable cache evict করা যাবে না। তবে cache affinity-এর জন্য starvation/biased puzzle selection করা যাবে না।

Candidate scheduler: `priority = α * estimated_cached_tokens_saved + β * useful_progress - γ * pool_pressure - δ * waiting_age_penalty`; `waiting_age_penalty` এখানে age বাড়লে priority **বাড়াতে** হবে—বাস্তবে formula implement করার সময় `+ δ * waiting_age` ব্যবহার করুন, বা `- δ * estimated_starvation` যেখানে sign ঠিক আছে। `/cache` marker/latency estimation না থাকলে affinity-কে hypothesis হিসেবে record করুন। Freeze `max_tasks`, `tokens/task`, deadline, total work এবং agent order fairness distribution; only scheduling policy vary।

CPU replay: recorded tool-call/return timestamps দিয়ে no-GPU simulate retained-prefix `value_i`, last-touch LRU, candidate eviction bytes, task finish time। GPU A/B-এ cached tokens/turn, re-prefill, live streams, solver calls এবং correct ARC outputs/hour compare। **Stop:** one task over-prioritized, deadline drain harmed, fairness breach, inconsistent scoring budget or cache-state corruption।

Y8. Real 1500+ objective: three denominators, measurable improvement and 12-hour economics

- `saturated_decode_tps = accepted_output_tokens / stable_active_decode_seconds`
- `mixed_serving_tps = accepted_output_tokens / entire_serving_window_seconds` (includes prefills/queues)
- `agent_e2e_tps = correctly generated output tokens / full-agent_wall_seconds` (tools/deadlines included)
- `quality_adjusted = correctly_submitted ARC queries / GPU-hour`

Example: 240×64k output tokens = 15.36 million. With startup+finalization >0, one pass at 700 useful tok/s takes ~6.10 h just generation; two passes require ~12.19 h **before overhead**. At 1500 useful tok/s, two 64k passes' generation would take ~5.69 h (not a predicted Kaggle result). All denominators must come from actual timestamps; unused drafts are never accepted output.

`throughput` A/B PASS only if (a) no correctness regression, (b) exactly same scored work budget, (c) 40k/90k performance improvement beyond noise, (d) actual resident count and missing data reported, (e) offline Kaggle dependencies, warmup and final submit path tested। Keep failed experiments, do not silently discard rows.

Y9. Competition knowledge transfer (not model-specific prescription)

AIMO-2 4×L4 7B/14B benchmark and 7th-place scheduling write-up: reproduce batch-size/context trade-off method, budget-adaptive task routing, and explicit GPU-memory headroom—not their exact model TPS. Gemma 4 Developer Agent: warmup graph OOM and no-network wheelhouse engineering. ARC-AGI-2/3: Duck agent lease, tool-turn batching, deadline tails. All are useful implementation mining sources.

https://www.kaggle.com/competitions/ai-mathematical-olympiad-progress-prize-2

https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2

https://www.kaggle.com/competitions/gemma-4-developer-agent

https://github.com/vllm-project/vllm/pull/60532

https://github.com/vllm-project/vllm/pull/61224

Z. v4 audit ledger — তৃতীয় AI গবেষণার পূর্ণ coverage এবং source status

Source finding	Coverage / decision	Evidence type
Decode vs sustained vs agent throughput denominators	Y8; existing G	[METHOD]
Franzen checkpoint LRU/retention, low-M GEMM, history trim	Y6/Y7 + separate SGLang v4	[UPSTREAM]
vLLM #53504 first repeat misses; #52244 alignment	Y2	[LEAD; exact PR #52244 verify]
#53670 last-block drop, multi-turn loss	Y2, V4-05	[UPSTREAM; gated]
vLLM Mamba #57111, #55697	Y3, V4-06	[UPSTREAM/RFC]
SGLang #40865 retreat 93%, ~30% compute	Y3, separate SGLang v4	[UPSTREAM]
L4 llama.cpp MMVQ verification 16.2%	Y6, V4-11	[UPSTREAM]
TP logits sampling and local argmax	Y4/Y6; compare v3 V32-01	[VERSIONED]
NCCL and GDN kernel/attention cost	Y6; v3 + SGLang ledger	[PROFILE PENDING]
Prefill/Decode mixed case #54919, eager myth #51008	Y5	[UPSTREAM]
Session lease, fairness and task affinity	Y7	[EXPERIMENT]

Source finding	Coverage / decision	Evidence type
Dynamic concurrency and actual running N	Y1/Y8; old C/G	[LOCAL]
MTP depth + target parity; precision changes	old D and W, Y1/Y2	[LOCAL+UPSTREAM]
Duck 93.43% prompt reuse, 836.6 P90 (not 6h average)	v2/v3 historical sources retained	[UPSTREAM]
Flash-Next/other frameworks	existing P3, only transfer kernel ideas; model outside primary scope	[SCOPE]
12-hour token economics and startup limit	Y8	[ARITHMETIC]
NVIDIA Team 1500+ reference target	Cover, Y1/Y8	[USER-SPECIFIED]

Conflicts reconciled: (1) The new AI suggests block-drop first; our first tests keep lower-risk prefix-match and argmax first, with block-drop separately **because cache-state corruption is a known failure class**. (2) `#52244` and `#55697` need exact source/merge-state verification before backport. (3) `#51008` invalidates the popular blanket claim that uniform MTP always executes GDN eagerly. (4) Other GPU results supply methods/patches, not an imposed throughput ceiling; L4 adaptation is the point of experimentation.

v4 non-regression audit contract: retain all v3 headings and all old full URLs; v4-source findings each map to a section above; every new experiment has a control, a measurable output and STOP/GATE; generated PDF must preserve Bengali glyphs and contain clickable literal GitHub/Kaggle/HF URLs. No model notebook or GPU submission was changed to produce this research update.

AA. Final integration v5: Claude E22 ও পাঁচটি গবেষণার সমন্বিত সিদ্ধান্ত

সংস্করণ: 29 October 2026, final research synthesis. **Target:** Qwen3.8-27B, Kaggle 4×L4, sustained accepted-output **1500+ tok/s**, quality preserved. এখানে আগের তিন GPT research, আমাদের স্বাধীন research dossier (v1–v4) এবং Claude-এর `Q27_E22_speedlab_vLLM032_SGLang_2026-10-29.zip` মিলিয়ে **নতুন priority order** নির্ধারণ করা হলো। আগে যে দুটি engine-কে “baseline” বলেছি, তা কেবল reproducible controls: inference version, quantization, TP/PP, draft algorithm, cache policy, source code, custom kernel সবই **পরিবর্তনযোগ্য experimental variable**। আলাদা branch, positive control, cold-start reinstall এবং quality gate ছাড়া কোনো winner production-এ যাবে না। NVIDIA Team-এর 1500+ user-supplied target premise পুনর্বিচার করিনি।

Evidence convention: `[LOCAL RAW]` = সংরক্ষিত engine/NCCL log/JSON থেকে পুনরায় পড়া; `[LOCAL REPORT]` = supplied measured report, এই session-এ GPU rerun নয়; `[UPSTREAM]` = GitHub/HF/Kaggle source-এর report; `[CLAUDE MODEL]` = Claude-এর analytical forecast/fit; `[PLAN]` = এখনও চালানো হয়নি; `[VERIFY]` = source compatibility/PR status unresolved। **E22 notebook দুটি experiment পরিকল্পনা; completed E22 GPU benchmark নয়।**

AA1. প্রথমে নতুন top priorities: বাস্তবায়ন ও success gate

Rank	Experiment ID	Exact change / hypothesis	Minimum test; promote if	Risk/cost
P0-01	`V32-AR-TRACE`	TP4 NCCL critical-path profiling, 983040B × 140 all-reduces ও ~11.9MB all-gather; CUDA graph capture	343.88 µs measured baseline; matched 40k/90k actual GPU trace and output TPS	Low; measurement
P0-02	`V32-ARGMAX`	MTP3 `use_local_argmax_reduction=true`; sharded greedy draft-token choice	No token-ID drift; gather kernel/time reduction; 8k/40k/90k	Low–medium
P0-03	`V32-GRAPH`	Existing `TRITON_ATTN` FULL graph vs FlashInfer PIECEWISE; selective #60532 full-graph backport	Log proves actual captured multi-token verify, no graph deadlock/OOM, 90k speed	Medium patch
P0-04	`V32-HYBRID-REUSE`	`prefix_match_unit` 64/32/16 then shared Mamba checkpoint; each independently	real 3–5 turn tool replay: generated-tail cache hit and total re-prefill time ↓	Medium correctness
P0-05	`V32-INT4-ORACLE`	FP8 vs INT4 identical task IDs; target-only & MTP3 under same Duck	Genuine quality gap quantified; no first-token corruption, no systematic high-margin flip	Medium
P1-01	`V32-CTA`	NCCL default vs CTAs 2/4/8, protocol/thread controls	Net verify-step p50/p95 ↓, correctness and 90k capacity unchanged	Low–medium
P1-02	`V32-BF16-SSM`	float16 → bfloat16 / default GDN state, identical weights	Fused CUDA GDN selected, vs current Triton fallback; enough streams still fit	Low
P1-03	`V32-ACTIVE-POOL`	`max_num_active_seqs`, actual KV bytes, concurrency 12/16/20/24, context 8k–130k	resident streams × accepted/step improvement without re-prefill/preempt cost	Medium
P1-04	`V32-BLOCKDROP`	isolated `disable_eagle_block_drop=true`	first-repeat cached tail ↑, forced rejection/cache-state parity PASS	**High numerical**
P1-05	`V32-PREFILL`	2048/4096/8192 batched tokens; 19 warm+1 cold 90k, staggered tools	mixed-agent useful token/hour ↑, no decode starvation	Low–medium

Rank	Experiment ID	Exact change / hypothesis	Minimum test; promote if	Risk/cost
P1-06	`V32-INT4-FULL`	stock INT4 MTP3 + graph + NCCL winner combined	single-arm gains independently replicated first; same ARC quality	Medium
P2-01	`V32-HSAR-BF16`	**new four-rank host-staged all-reduce**; two-shot, mapped pinned host memory, graph-capable	per-message correctness vs `all_reduce` oracle; ≤250 µs/call before integrating	**High engineering**
P2-02	`V32-HSAR-INT8`	Compress collective payload to INT8 + explicit scaling and reconstruction	sensitive logits/high-margin/ARC gates, saturated and wall TPS gain	**High numerical**
P2-03	`V32-NVFP4-KV`	0.32 `nvfp4` path / compatible patch, possible KV density boost	actual SM89 reader/ writer, head_dim256, FP8-oracle accuracy, 90k speed	**High**
P2-04	`V32-ADA-FP8`	SM89-native W8A8 block-FP8, PR #61224, prebuilt CUDA kernels	speed microbench for actual Qwen3.8 GEMM shapes; whole-model correctness	**High build**
P2-05	`V32-MAMBA-DECODE-CKPT`	#50551 finalized assistant-reply checkpoint (if available/ backportable)	5-turn generated-tail reuse; no stale state or hidden score loss	**High correctness**
P3	`V32-PP-MERLIN`	PP4/TP2×PP2, 4×TP1 with specially packed checkpoint; Merlin/ low-M GEMM/ FlashInfer plan-cache	startup in 10m, 50k–130k cache fit, matched quality, then throughput	Architectural R&D

এই table **experiment order**, predicted speedup ranking নয়। GPU quota কম হলে আগে P0 source/CPU and 8k smoke, তারপর 40k, সবশেষে 90k/130k ও full agent। কোনো arm-এ random answer বা high-margin token flip দেখলে speed কতই হোক reject।

AA2. Claude-এর সবচেয়ে শক্ত evidence: আমাদের NCCL measurement আবার পড়া

`engine_pack/measurements/E14A_nccl_bench.json`-এ `fake=false`, চার L4, NCCL 2.29.7, PCIe PHB, `torch 2.13.0+cu130`, CUDA 13.0 এবং log অনুযায়ী GPU max PCIe link gen 3। Kaggle pinned notebook torch 2.10/ cu128; **benchmark venv ও actual engine venv এক করে দেখাবেন না**। NCCL 983,040-byte message/ 140-call CUDA-graph replay:

Physical communication mode	Graph per all-reduce	Eager 983,040 B	Result
Default NCCL SHM	**343.8848 µs**	381.952 µs	**best measured**

Physical communication mode	Graph per all-reduce	Eager 983,040 B	Result
`NCCL_P2P_LEVEL=SYS` / P2P IPC	**1880.5212 µs**	1938.944 µs	**5.47× slower**
`NCCL_SHM_USE_CUDA_MEMCPY=1`	Graph **timeout**	502.784 µs	NO-GO

Source: `Q27_E22_speedlab_vLLM032_SGLang_2026-10-29.zip/engine_pack/measurements/E14A_nccl_bench.json`। `nvidia-smi topo -p2p`-তে peer access **OK** হলেও fast peer transport প্রমাণ হয় না: functional P2P সম্ভব, কিন্তু এই VM-এ measured performance খারাপ। FlashInfer PCIe-IPC-ও আগের VMS পরীক্ষায় 0.68×। **নতুন idea** custom host-staged collective; এটি current measured winner নয়।

Claude-এর E14A reported SGLang trace (W8, 12 streams, ~5.9k context, 96 verify rows) 116.1 ms kernel sum = 50.9ms all-reduce (140 calls) + 8.1ms all-gather (3 calls) + প্রায় 57ms অন্য kernels। সতর্কতা: এটি **SGLang-specific**। vLLM-এ 59ms NCCL একইভাবে measured হয়েছে—এমন দাবি নয়; vLLM-এ নিজস্ব profiler নিতে হবে। Cumulative kernel durations true non-overlapped critical path কি না trace dependency দিয়ে দেখতে হবে।

Implementation-guided host-staged AR: উৎস SGLang no-P2P staged communication PR #39605; vLLM-এ independent drop-in হবে না। Design: rank-local CUDA→pinned mapped host buffer, control/signalling or graph-safe stream protocol, ring/two-shot reduction, host→CUDA broadcast; প্রতি rank-এ completion/data correctness ও stream lifetime trace। TP4 native group semantics, all-reduce order, tensor dtype, alignment, CUDA graph capture, Kaggle security/IPC restrictions সব আলাদা যাচাই। Stage 1: no capture and exact BF16/FP32 oracle; stage 2 graph capture; stage 3 model integration. **Success target `≤250 µs` per 983KB call একটি engineering go threshold, measured performance নয়।** BF16 AR speed projection +14–17% এবং INT8 +30–35% Claude-এর model estimate; user-এর Qwen3.8-এ কোনো new measured result নয়। INT8 communication inference distribution বদলাতে পারে; strict acceptance/long-context benchmark ছাড়া “lossless” দাবি করবেন না।

https://github.com/sgl-project/sglang/pull/39605

https://github.com/NVIDIA/nccl-tests

https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html

AA3. Full CUDA graph—দুটি আলাদা উপায় ও একটি real failure mode

0.32 stock FlashInfer + MTP3: engine log-এ `FULL_AND_PIECEWISE ... not supported ... FlashInferBackend (UNIFORM_SINGLE_TOKEN_DECODE)`, ফলে effective `PIECEWISE`। E22 notebook already plans `TRITON_ATTN` + requested FULL, কিন্তু **notebook present ≠ successful graph capture**। Compare:

```
G0  FlashInfer + MTP3 + current PIECEWISE, unchanged
G1  TRITON_ATTN + MTP3 + FULL_AND_PIECEWISE
G2  TRITON_ATTN + MTP3 + FULL_DECODE_ONLY (if G1 incompatibility)
G3  FlashInfer + PR #60532 selected minimal backport, FULL_AND_PIECEWISE
```

#60532-এর upstream SM80 TP1/2 MTP results 73.6→108.1 (47%) এবং 61.0→83.1 (36%) reported; **our SM89 TP4 16–24 long streams-এ gain অজানা**। Backport অবশ্যই source path, backend capability declaration, q_len and graph-safe verify kernel একসঙ্গে নিতে হবে; `cudagraph_mode` flag alone enough নয়।

Verify actual graph replay counts, CUDA graph allocated bytes, 90k KV pool shrink, graph/eager token-ID comparisons, forced MTP reject and tool-return state। `#42271` MTP/full-graph deadlock reference থাকলে separate stress test। 8k speed-only winner 90k saturated workload-এ loser হতে পারে।

https://github.com/vllm-project/vllm/pull/60532

https://github.com/vllm-project/vllm/issues/42271

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/config/compilation.py

AA4. Sharded MTP sampling—কতটা communication সরানো যায়

Current `MarlinFP8ScaledMMLinearKernel` FP8 block-128 weights কে W8A16 path-এ চালায়। `use_local_argmax_reduction` সম্পূর্ণ vocabulary logits জড়ো না করে per TP rank local winner reduce করার mechanism; exact `SpeculativeConfig` model-path compatibility verify। `O(vocab)→O(TP)` reduction claim **draft greedy selection**-এর জন্য, সব TP all-reduce-এর নয়। Greedy ties, rank offset/index, logits masking, top-p/temperature/non-greedy না বদলিয়ে input IDs/seed matched test করুন। যদি draft model path option ignore করে, effective activation log না থাকলে claim PASS নয়।

```
{"method":"mtp","num_speculative_tokens":3,"use_local_argmax_reduction":true}
```

Control G0 current MTP3; only option change in test G4। vLLM #59548-এর reported PIECEWISE vs eager ~1.01x অনেক batch-এ; তাই graph patch ও local argmax-এর gain নিজের workload-এ measurable হওয়া জরুরি।

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/config/speculative.py

https://github.com/vllm-project/vllm/issues/59548

AA5. NVFP4 KV—capacity gain হতে পারে, কিন্তু exact release/PR implementation যাচাই

Claude report NVFP4 `--kv-cache-dtype nvfp4`-কে v0.32 installed capability বলেছে; current versioned CLI-তে dtype listed থাকলেও **source-code compilation/runtime path আলাদা বিষয়**। GitHub PR #46963 direct page 2026-10-29 retrieval-এ Open দেখা গেছে, অন্য release summary-তে SM8x support claimed। তাই **merge status/packaged code discrepancy** unresolved; version-pinned `v0.32.0` wheel-এর actual backend and needed FlashInfer APIs inspect করে fix করুন। Dtype parser accept করলেই SM89 GDN long decode works—এমন নয়।

Theoretical FP8 attention KV per TP4 GPU: 16 attention layers × K/V × 1 KV head × 256 dim × 1 byte = 8192 bytes/token; 24 independent ×130k = 25.56 GiB per GPU KV alone, 24GiB L4-এর উপর। NVFP4 representation হয়তো ~1.75× more KV tokens (packing+scales accounted) দিতে পারে; এটি **benchmark-dependent**, no universal 2×। Pre-SM100 fallback may dequantize KV to BF16 for FA2; SM89 bandwidth/latency, head_dim256, MTP verify graph coverage আলাদা validate করতে হবে। CPU offload idle sessions-এ কাজ করতে পারে; active per-token KV accesses CPU offload করলে PCIe bottleneck আরও বাড়তে পারে। `#58653`, `#63565`-এর hybrid-tier caveats নোট করুন।

```
N0: same INT4/FP8 model + fp8_e4m3 KV (control)
N1: only nvfp4 KV (must actually boot and generate)
N2: N1 + safe active concurrency increase, only after 8k/40k quality
```

Test 32k/90k/130k context, long sampled reasoning, logprob/no high-margin flips, TP4 accepted output, actual KV pool count and full graph mode. Related PR on RTX3090 reported ~3× relative to BF16 auto—not the same as 1.75× relative to **your existing FP8 KV**. Treat Claude's NVFP4+32-stream predictions (XL 500–560 with host-staged AR) as **model forecasts only**.

https://github.com/vllm-project/vllm/pull/46963

https://github.com/vllm-project/vllm/issues/47684

https://github.com/vllm-project/vllm/issues/58653

https://github.com/vllm-project/vllm/issues/63565

https://docs.vllm.ai/en/v0.32.0/cli/serve/

AA6. Existing cache research + Claude checkpoint update: cumulative savings across turns

Old verified source-backed ideas retained: vLLM #53504 first repeated prompt misses; #53670 EAGLE last-block drop & 1600/1648-token re-prefill; #52244 block hash and last-token offset; partial-prefix PR #46384; `--prefix-match-unit`, `--enable-mamba-shared-prefix-checkpoint`; #50551 finalized assistant-tail decode checkpoint; #57111 retreat; #55697 application-directed checkpoints. Claude adds capacity calculations and a strong production qualification: vLLM align mode may cache prompt-tail but not the generated reply, making subsequent agent turns re-prefill assistant output. **Do not conflate shared-prefix checkpoint with finalized generated-tail checkpoint:** two different state locations.

Best isolation:

Arm	Only delta	Diagnostic question
H0	Default align and current prefix	first-repeat hit?
H1	`--prefix-match-unit 64`	finer hash improves actual match?
H2	H1 + shared-prefix checkpoint	branch junction state available?
H3	H1/H2 with unit16 if valid	smaller unit worth overhead?
H4	H2 + `disable_eagle_block_drop=true`	last block retained correctly?
H5	finalized decode checkpoint #50551 backport	previously generated assistant reply reused?

`prefix_match_unit` must divide each resolved physical cache group; 64/32/16 choice should be made from logs, **not from just LCM**. Same exact system/task prompt→10k thinking→Python tool→tool response→20k thinking→next turn repeated; 8k/40k/90k history progression; request-level `cached_prompt_tokens`, `uncached`, `mamba_checkpoint_id`, `last_cached_generated_position`, `prefill_wall`, `effective_running`. A flag is no gain if it only increases reported cache hits without saved GPU time. Last-block disabling is **high-risk state correctness**, never combine before independent numerical tests.

https://github.com/vllm-project/vllm/issues/53504

https://github.com/vllm-project/vllm/issues/53670

https://github.com/vllm-project/vllm/pull/46384

https://github.com/vllm-project/vllm/pull/50551

https://github.com/vllm-project/vllm/issues/57111

https://github.com/vllm-project/vllm/issues/53912

AA7. Active-stream residency, queueing, tail occupancy—120k history-তে outcome বদলায়

Current measured XL arms: vLLM INT4 requested20→actual running12, vLLM FP8 requested16→actual11; SGLang requested20→actual7। Model-dependent 90k degraded response-কে শুধু "CUDA slow" বলে ব্যাখ্যা করা যাবে না। Two measurements: (1) stable resident decode TPS and actual active count; (2) full mixed agent wall TPS with tool returns, cold prefills and tail starvation।

```
A: max_num_seqs=16 / active_seq=12,16   (capacity matched)
B: max_num_seqs=20 / active_seq=16,20   (matched work)
C: max_num_seqs=24 / active_seq=20,24   (only if KV/Mamba fit)
```

Turn-aware CPU scheduling idea: short tool runs finishing soon may get a bounded session lease before cache eviction; don't pin forever and starve fresh puzzles. Improve `correct ARC queries/GPU-hour`, not just `max-num-seqs`. Preemptions/recompute ratio, waiting tasks, cache expiry, tool wall-time, program validity and deadlines must be reported. Change `--max-model-len` to 139264 only after 130k+output reserve capacity check and cold-start graph-memory profiling; a global model-len change can alter the memory pool without improving 40k TPS.

AA8. Compatibility ledger: hypotheses that cannot be silently promoted

Candidate	What is available	Why not automatic production
Ada W8A8 CUTLASS #61224	L4 Qwen3-8B author +42% at batch; vLLM0.32 default Marlin W8A16	model shapes, compiler/CUDA/torch ABI, FP8 scale contract unverified
BF16 GDN state	fused CUDA GDN spec-decode eligible; FP16 uses Triton	fused CUDA may be slower and reduce admitted streams; match budgets
Merlin W4A16 TP1/TP4	checkpoint packs lm_head/MTP into INT4, host embeddings; patch series	checkpoint itself different; exact oracle quality, startup under 10m
PP4/TP2×PP2	may reduce per-layer TP collectives	vLLM hybrid PP/MTP source support + illegal-access reports need gate
DBO / two-batch overlap	vLLM option	dense GDN TP4 MTP path gain unknown; possible state race
FP8 scales	logs may say default 1.0	verify per-layer scale materialization before attributing quality drop
Spec width2/4/5	previous MTP3 beat MTP2/4	revisit only after verification kernel changed / high-concurrency adaptation

Candidate	What is available	Why not automatic production
All P2P/FlashInfer IPC AR	already 0.68× or worse	not profitable on measured current topology

https://github.com/vllm-project/vllm/pull/61224

https://github.com/vllm-project/vllm/issues/63505

https://github.com/vllm-project/vllm/pull/46994

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/config/cache.py

AA9. Experimental design—E22 notebooks কী আছে এবং কী নেই

`Q27_E22_speedlab_vLLM032_SGLang_2026-10-29.zip`-এর `e22_notebooks/Q27-XHIGH-E22_SPEEDLAB_VLLM032.ipynb`-এ code-cell executable engine matrix আছে: FP8 ref/oracle, `TRITON_ATTN` requested FULL, INT4 s20/s24, CTA variations, bigger prefill and combination arms। **এই ZIP-এ নতুন E22 per-arm result JSON বা accepted speed outcome নেই**। E22 launch success/throughput improvements pending; notebook code ≠ confirmation. Claude-এর modeled 8k 850–900/40k650/XL450 ইত্যাদি **forecasts**, 1500+ achieved নয়।

Three bounded campaigns:

1. **CPU/source preflight:** v0.32 ABI; exact flags and effective resolved configs; builder/notebook SHA; offline wheelhouse and CPU log analysis; validate no `fake` bench mixed with GPU data.
2. **Short independent GPU arms:** 3–5 minute matched 8k/40k windows for argmax, FULL graphs, NCCL CTAs, cached-turn tests; one knob per engine process; record graph actual modes and model hashes.
3. **90k/130k + sampled agent replay:** only winning arms, 20–30 min; `actual_running`, accepted output, stable vs full-wall, tool behavior, generated-tail checkpoint provenance, accuracy and deadline, clean fallback and startup ≤10 min.

Three mandatory denominator metrics: `stable_decode_tps`, `mixed_serving_tps`, `agent_e2e_tps`. Count only emitted accepted generated tokens, not discarded drafts. ARC metric `correctly_submitted_query_count/GPU_hour`; equal evaluation tasks, request IDs and budgets. Confidence intervals or repeated A/B randomization where GPU time permits; missing samples → `INCONCLUSIVE`, never `PASS`.

AA10. Source conservation / five-study audit map — vLLM

Research source	Unique important insight retained	Where in this PDF
GPT-1 `Pasted text(9).txt`	Merlin/TP1×4, EAGLE-4, adaptive speculation, hybrid prefix repairs, GEMM/graph backports, accuracy gap, sampling-budget math	Previous W/V sections; AA8
GPT-2 `Pasted text(10).txt`	BF16 SSM eligibility; #60532 FULL graphs; #61224 SM89 native FP8; #42917 cache PPL; E14-style workload and Kaggle AIMO2	Previous V3 X sections; AA3/AA8

Research source	Unique important insight retained	Where in this PDF
GPT-3 `Pasted text(20261029-065936).txt`	v0.32 local argmax, Mamba split/ checkpoint, #36548/41736/41737/43029, NCCL no-P2P path, custom TP all-reduce	Previous V3/Y; AA2/AA4/AA6
Independent GPT Research + v1–v4	8k/40k/XL matrix, staged-cache gate, long-prefill starvation, replay state/ ReplaySSM risk, Duck fairness, offline startup, exact URLs	All preexisting headings + AA1–AA8
Claude E22 engine+5 notes	Real NCCL raw µs, 116ms decomposition, graph+HSAR+NVFP4 proposals, model forecasts, 3 candidate configurations, tail occupancy, fixed-shape allgather cost	AA1–AA9; remaining SGLang-specific details in separate v5 PDF

Explicit conflicts resolved: (a) Claude puts AR at top because communication dominates its measured SGLang step; vLLM-specific AR fraction remains unknown. (b) Claude calls NVFP4 PR #46963 merged; public PR page still shows Open while v0.32 docs list dtype: wheel implementation must be examined, no silent conclusion. (c) Memory saving is not quality saving. (d) 1500+ target is a motivating design constraint, not an achieved result in this ZIP. (e) Merlin/3090/Intel data are **implementation recipes to adapt**, not reasons to abandon research.

Zero-omission principle: Prior v4 material retained verbatim except version navigator. Link/ reference audit must confirm every old heading and every prior literal URL appears in the new v5 PDF source and extracted PDF text. No GPU execution or notebook mutation was performed by this document update.

Version 5.0 · এই PDF পুনরায় আপডেটযোগ্য; পরবর্তী AI গবেষণা যোগ করার সময়ে প্রমাণ, experiment ID ও revision history সংরক্ষণ করতে হবে।